

---

## Guía paso a paso — Portal de Turismo “Descubrí Santa Cruz”

**Asignatura:** Laboratorio Web 1 — Tecnicatura Universitaria en Desarrollo Web.

**Profesor:** Jorge Ceferino Valdez.

**Público objetivo:** Alumnos con introducción previa a HTML5, CSS3 vanilla y JavaScript vanilla que desean consolidar sus conocimientos mediante un proyecto práctico y progresivo.

**Tecnologías:** HTML5, CSS3 vanilla, JavaScript vanilla (sin frameworks ni librerías).

**Enfoque pedagógico:** Cada tarea se apoya en la anterior, permitiendo ver la evolución de una “página estática” hasta una “aplicación web interactiva”. Se introducen patrones de maquetación responsive (Mobile First), variables CSS, accesibilidad (ARIA), y un modelo de estado + renderizado en JavaScript para filtrar contenido dinámicamente.

---

### Tabla de contenidos

1. Visión general del proyecto
2. Cómo leer esta guía
3. Tarea 1 — Estructura HTML: Header, Hero y ¿Por qué Santa Cruz?
4. Tarea 2 — CSS base, variables y estilos del Header + Hero
5. Tarea 3 — Sección Destinos: tarjetas con imágenes y badges
6. Tarea 4 — Secciones Testimonios y Newsletter
7. Tarea 5 — Footer y Formulario de Contacto
8. Tarea 6 — JavaScript: Menú hamburguesa móvil
9. Tarea 7 — JavaScript: Filtrado y búsqueda de destinos
10. Tarea 8 — JavaScript: Validación del formulario de contacto
11. Tarea 9 — JavaScript: Newsletter y navegación por hash
12. Tarea 10 — Responsive design y modo oscuro
13. Arquitectura final: decisiones clave
14. Checklist de aprendizaje

---

### 1. Visión general del proyecto

Este ejercicio construye un portal de turismo provincial funcional en el navegador. Al finalizar las 10 tareas, la aplicación permite:

- Visualizar una landing page con 8 secciones: Header, Hero, ¿Por qué Santa Cruz?, Destinos, Testimonios, Newsletter, Contacto y Footer.
- Filtrar destinos turísticos por categoría (Naturaleza, Aventura, Cultura, Gastronomía) con actualización instantánea.
- Buscar destinos por nombre con normalización de acentos.
- Navegar entre secciones por hash sin recargar la página (client-side routing básico).
- Validar un formulario de contacto con feedback visual y accesible (ARIA).
- Suscribirse a un newsletter con feedback dinámico.
- Usar un menú hamburguesa en móvil con transiciones animadas.

- Adaptarse automáticamente a modo oscuro del sistema operativo.

El proyecto se compone de tres archivos de código, uno por cada **capa** de una página web:

| Archivo        | Capa             | Responsabilidad                                                     |
|----------------|------------------|---------------------------------------------------------------------|
| index.html     | Estructura       | Qué elementos existen, su semántica y su jerarquía                  |
| css/styles.css | Presentación     | Cómo se ven esos elementos, layout responsive y modo oscuro         |
| js/main.js     | Conducta y datos | Qué hace la página: interacciones, filtrado, validación, navegación |

La idea central de la cursada es que cada tarea **agrega una capacidad** sin romper lo anterior. El sitio es usable desde la Tarea 1 (visualmente); lo que cambia es cuánto puede hacer.

## 2. Cómo leer esta guía

Cada tarea sigue siempre el mismo esquema:

1. **Objetivo** — qué problema concreto vamos a resolver.
2. **Pasos** — el código, presentado en el orden en que conviene escribirlo.
3. **Decisiones de implementación** — el porqué de **cada línea**. Esta es la parte más importante: copiar código es fácil; entender por qué está escrito así es lo que se evalúa.

Una recomendación de método: **escribí el código vos mismo**, no lo copies y pegues. Al teclear cada línea te vas a hacer preguntas (“¿por qué data- categoria y no una clase?”, “¿por qué clamp() y no media queries para la fuente?”) y esta guía las responde justo al lado.

A partir de la Tarea 7 vas a ver una técnica nueva: **estado + renderizado**. Primero escribiremos el código “suelto”, luego detectaremos la repetición y finalmente lo organizaremos en variables de estado y una función de filtrado. Esa secuencia “primero el problema, después la solución” es deliberada.

## 3. Tarea 1 — Estructura HTML: Header, Hero y ¿Por qué Santa Cruz?

### Objetivo

Crear la estructura semántica del documento HTML, incluyendo el encabezado con navegación, el banner principal (Hero) y la primera sección de contenido con tarjetas de razones para visitar Santa Cruz.

### Paso 1.1: Estructura base del documento HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Descubrí Santa Cruz – Portal de Turismo Provincial. Explorá glaciares, montañas, cultura y gastronomía de la Patagonia Argentina.">
  <title>Descubrí Santa Cruz – Portal de Turismo Provincial</title>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;700&family=Source+Sans+3:wght@300;400;600;700&display=swap"
    rel="stylesheet">

  <link rel="stylesheet" href="css/styles.css">
</head>
```

```

<body>
  <!-- El contenido irá acá -->
  <script src="js/main.js"></script>
</body>
</html>

```

## Decisiones de implementación, línea por línea:

- **<!DOCTYPE html>**: Le indica al navegador que use el modo estándar de HTML5. Sin esta línea, algunos navegadores entran en “quirks mode” (modo de compatibilidad antiguo) y el CSS se interpreta de forma inconsistente.
- **<html lang="es">**: El atributo lang declara el idioma del contenido. Mejora la accesibilidad (los lectores de pantalla eligen la voz correcta para la síntesis de voz) y el comportamiento del navegador (corrector ortográfico y traducción automática).
- **<meta charset="UTF-8">**: Define la codificación de caracteres. Garantiza que tildes, ñes, emojis y símbolos matemáticos se rendericen correctamente. **SIN ESTO**, los textos en español se verían mal (con caracteres rotos como Ã± en lugar de la ñ). Va lo más arriba posible dentro del <head> para que el navegador sepa cómo decodificar todo el contenido desde el inicio.
- **<meta name="viewport" content="width=device-width, initial-scale=1.0">**: Metaetiqueta esencial para diseño responsive. Le dice al navegador que el ancho de la ventana de visualización (viewport) debe coincidir con el ancho del dispositivo físico a escala inicial 1.0. **SIN ESTO**, los teléfonos móviles muestran la página “achicada” como si fuera una pantalla de escritorio de gran tamaño, obligando al usuario a hacer zoom manual. Es obligatoria para diseño responsive / mobile-first.
- **<meta name="description" content="...">**: El texto resumido que aparece debajo del título en los resultados de búsqueda de Google (SERP). También lo usan las redes sociales al compartir el enlace. Debe ser descriptivo, atractivo y contener palabras clave relevantes.
- **<title>**: Texto que aparece en la pestaña del navegador. No es decorativo: es el título oficial del sitio web, se utiliza para guardar marcadores (favoritos), en el historial de navegación y como enlace principal en los resultados de Google.
- **<link rel="preconnect" href="https://fonts.googleapis.com">**: “Pre-conecta” con el servidor de Google Fonts ANTES de que el navegador lo necesite en el flujo de renderizado. Ahorra tiempo de conexión DNS + TCP. El atributo crossorigin en la segunda pre-conexión es necesario porque el servidor de fuentes externas es un origen diferente al nuestro.
- **<link href="...fonts.googleapis.com/css2?family=..." rel="stylesheet">**: Descarga la hoja de estilos de Google Fonts. Usamos Playfair Display (serif elegante) para títulos y Source Sans 3 (sans-serif moderna) para cuerpo de texto. El parámetro &display=swap muestra primero el texto con una fuente del sistema y “cambia” a la fuente descargada cuando esté lista, lo que evita que el texto sea invisible durante la carga de red (mejorando la experiencia del usuario).
- **<link rel="stylesheet" href="css/styles.css">**: Vincula la hoja de estilos **externa**. La ruta es relativa a la ubicación del archivo HTML actual. Mantener el CSS separado garantiza la separación de responsabilidades. ⚠ **ORDEN DE CARGA**: Los estilos se aplican en cascada. Si se declaran múltiples hojas de estilo, la última declarada tiene prioridad sobre las anteriores (a igualdad de especificidad).
- **<script src="js/main.js"></script> al final del <body>**: El navegador lee el HTML de arriba a abajo. Si el <script> estuviera en el <head>, se ejecutaría **antes** de que existan los elementos del DOM que quiere manipular, lo que provocaría errores de “elemento no encontrado” al usar funciones como getElementById. Colocándolo justo antes de la etiqueta de cierre </body> garantizamos que todo el DOM ya está construido cuando el script corre.

## Paso 1.2: Header y Navegación

```

<header class="header" id="header-principal">
  <div class="contenedor header-contenido">
    <a href="index.html" class="logo" aria-label="Ir a la página de inicio">
      <span class="logo-icone"><img alt="Logo icon" data-bbox="288 745 305 758"/></span>
      <span class="logo-texto">Descubrí <strong>Santa Cruz</strong></span>
    </a>

    <button type="button" class="menu-hamburguesa" id="btn-menu" aria-label="Abrir menú de navegación"
      aria-expanded="false">
      <span class="hamburguesa-linea"></span>
      <span class="hamburguesa-linea"></span>
      <span class="hamburguesa-linea"></span>
    </button>

    <nav class="nav-principal" id="nav-principal" aria-label="Navegación principal">

```

```

<ul class="nav-lista">
  <li><a href="index.html" class="nav-enlace activo" aria-current="page">Inicio</a></li>
  <li><a href="#destinos" class="nav-enlace">Destinos</a></li>
  <li><a href="#contacto" class="nav-enlace">Contacto</a></li>
  <li><a href="#testimonios" class="nav-enlace">Testimonios</a></li>
</ul>
</nav>
</div>
</header>

```

### Decisiones de implementación, línea por línea:

- **<header>**: Etiqueta semántica de HTML5 que representa la cabecera de la página o sección. Generalmente contiene logo, navegación principal y elementos introductorios.
- **id="header-principal"**: Identificador único para referenciarlo o para anclajes. El header se fija con `position: sticky` en CSS para que “se pegue” arriba al hacer scroll. `z-index: 100` asegura que quede por encima del resto del contenido.
- **<div class="contenedor header-contenido">**: `.contenedor` es una clase de layout que centra el contenido y establece un ancho máximo. `.header-contenido` aplica Flexbox para alinear horizontalmente el logo, el botón hamburguesa y la lista de navegación.
- **<a href="index.html" class="logo" aria-label="...">**: Enlace del logo. Es una convención de usabilidad web que el logo **SIEMPRE** dirija a la página de inicio. El atributo `aria-label="Ir a la página de inicio"` anula el comportamiento por defecto de lectores de pantalla que leerían de forma confusa: “enlace, logo, montaña, Descubrí Santa Cruz”. Ahora leen directamente el propósito del enlace.
- **<span class="logo-icone">🏔️</span>**: El emoji es puramente decorativo. Al no aportar información textual que no esté en el propio logo, se incluye sin etiquetas alternativas ni descriptores ARIA para evitar ruido en lectores de pantalla.
- **<button type="button" ...>**: Define el botón hamburguesa móvil. Es obligatorio colocar `type="button"` de forma explícita; de lo contrario, algunos navegadores asumen por defecto `type="submit"` y recargan la página al hacer clic. `aria-expanded="false"` es un estado ARIA dinámico que indica a personas ciegas que el menú colapsable está cerrado. JavaScript lo cambiará a `"true"` al abrirlo. Las tres líneas internas se transformarán en una “X” usando animaciones CSS en la versión final.
- **<nav aria-label="Navegación principal">**: Define el bloque contenedor de la navegación principal del sitio. El atributo `aria-label` identifica este bloque semántico (muy útil si hay más de un `<nav>` en la misma página, como en el footer).
- **<ul>**: Los menús se estructuran como listas no ordenadas de enlaces por semántica: un menú es una lista de opciones. Los lectores de pantalla anuncian “lista de 4 elementos” facilitando la navegación interna.
- **href="#destinos"**: Enlace de anclaje interno. Apunta al id de la sección correspondiente (`id="destinos"`) logrando que el navegador desplace la pantalla hacia allí sin recargar la página.
- **aria-current="page"**: Atributo de accesibilidad que indica explícitamente que el enlace actual corresponde a la página en visualización.

### Paso 1.3: Hero Banner

```

<section class="hero" id="hero">
  <div class="hero-overlay">
    <div class="contenedor hero-contenido">
      <h1 class="hero-titulo">Descubrí Santa Cruz</h1>
      <p class="hero-subtitulo">
        Glaciares imponentes, montañas legendarias y paisajes que quitan el aliento.
        Tu próxima aventura patagónica empieza acá.
      </p>
      <a href="#destinos" class="btn btn-primario hero-cta">Explorar Destinos</a>
    </div>
  </div>
</section>

```

### Decisiones de implementación, línea por línea:

- **<section class="hero">**: El “Hero” es un patrón de diseño moderno que ocupa un gran porcentaje de la pantalla (70 % del alto o 70vh vía CSS) para captar la atención de inmediato mediante una imagen de fondo y un mensaje fuerte de llamada a la acción.

- **<h1>**: El título más importante del documento. **Solo debe existir un único <h1> por página web.**
- **.hero-overlay**: Capa superpuesta posicionada de forma absoluta (position: absolute) que cubre todo el banner. Su gradiente oscuro aplicado en CSS asegura que el texto blanco sea legible, independientemente de si la imagen de fondo es muy clara o muy oscura.
- **<a href="#destinos" class="btn btn-primario hero-cta">**: El Call to Action (CTA) es un enlace <a> y no un <button> porque su propósito semántico es **navegar** a otra sección (#destinos). Los botones <button> se reservan para **acciones** o eventos lógicos de JavaScript.

#### Paso 1.4: Sección “¿Por qué Santa Cruz?”

```
<main>
<section class="seccion por-que" id="por-que">
  <div class="contenedor">
    <h2 class="seccion-titulo">¿Por qué Santa Cruz?</h2>
    <p class="seccion-subtitulo">
      Una provincia única en el mundo, donde la naturaleza se expresa en su forma más pura.
    </p>

    <div class="tarjetas-grilla">
      <article class="tarjeta-razon">
        <span class="tarjeta-razon-icone" aria-hidden="true">🌿</span>
        <h3 class="tarjeta-razon-titulo">Naturaleza Salvaje</h3>
        <p class="tarjeta-razon-texto">
          Glaciares milenarios, lagos cristalinos y bosques patagónicos que te conectan con lo esencial.
        </p>
      </article>

      <article class="tarjeta-razon">
        <span class="tarjeta-razon-icone" aria-hidden="true">🧗</span>
        <h3 class="tarjeta-razon-titulo">Aventura Extrema</h3>
        <p class="tarjeta-razon-texto">
          Trekking, escalada en hielo, kayak y cabalgatas. El Chaltén es la capital nacional del trekking.
        </p>
      </article>

      <article class="tarjeta-razon">
        <span class="tarjeta-razon-icone" aria-hidden="true">👤</span>
        <h3 class="tarjeta-razon-titulo">Cultura Milenaria</h3>
        <p class="tarjeta-razon-texto">
          La Cueva de las Manos, Patrimonio de la Humanidad, guarda pinturas rupestres de más de 9.000 años.
        </p>
      </article>

      <article class="tarjeta-razon">
        <span class="tarjeta-razon-icone" aria-hidden="true">🍷</span>
        <h3 class="tarjeta-razon-titulo">Gastronomía Regional</h3>
        <p class="tarjeta-razon-texto">
          Cordero patagónico al palo, centolla fueguina, chocolates artesanales y cerveza craft de la región.
        </p>
      </article>
    </div>
  </div>
</section>
</main>
```

#### Decisiones de implementación, línea por línea:

- **<main>**: Etiqueta semántica de HTML5 que encapsula el contenido principal y exclusivo del sitio web. Los lectores de pantalla disponen de accesos directos para saltar directamente al <main> (skip link), omitiendo elementos repetitivos como el header. **Solo se permite un <main> por página** y nunca debe ser hijo de <footer>, <header>, <article>, <aside> o <nav>.

- **<section class="seccion por-que" id="por-que">**: <section> agrupa contenido temáticamente relacionado. La doble clase es una buena práctica de CSS modular: .seccion proporciona el padding común para estructurar el espaciado vertical de todas las secciones, mientras que .por-que define el color de fondo o propiedades únicas de este bloque.
- **Los <h2> y <h3>**: Los títulos de sección son <h2> para mantener la jerarquía de encabezados secuencial sin saltos (h1 → h2 → h3). Cada tarjeta de razón lleva un <h3> porque forma parte del contenido de su sección padre.
- **<article> vs <section>**: Usamos <article> para maquetar cada tarjeta individual de razón porque representa una unidad de contenido coherente, autocontenida e independiente que podría reutilizarse o distribuirse por separado.
- **aria-hidden="true" en los íconos**: Los emojis son DECORATIVOS. No aportan información textual que no esté ya en el título. Con aria-hidden="true", los lectores de pantalla los ignoran y no leen “hoja” o “escalador” al usuario.
- **Patrón de Tarjetas (Cards)**: Consisten en un patrón visual común: ícono/imagen + título + texto. En CSS se maquetan con Grid para reorganizarse de forma responsiva según el tamaño de la pantalla (1 columna en móvil, 2 en tablet, 4 en desktop).

## 4. Tarea 2 — CSS base, variables y estilos del Header + Hero

### Objetivo

Crear la hoja de estilos base con variables CSS (Custom Properties), reset universal, clases comunes de layout, y los estilos del Header sticky y el Hero banner.

### Paso 2.1: Variables CSS en :root

```
:root {
  /* — Paleta de Colores Patagónicos — */
  --color-glaciar: #2e86ab;
  --color-lago: #164e63;
  --color-atardecer: #f18f01;
  --color-rosa: #a23b72;
  --color-estepa: #c4a45a;

  /* — Colores de Superficie y Texto — */
  --color-fondo: #faf9f6;
  --color-superficie: #ffffff;
  --color-texto: #2b2b2b;
  --color-texto-secundario: #6b6b6b;
  --color-borde: #e0ddd5;
  --color-nieve: #f5f7fa;
  --color-cielo: #d6e8f0;

  /* — Tipografía — */
  --font-titulo: 'Playfair Display', Georgia, serif;
  --font-cuerpo: 'Source Sans 3', 'Segoe UI', system-ui, sans-serif;

  /* — Radios de Esquinas — */
  --radio-sm: 4px;
  --radio-md: 8px;
  --radio-lg: 16px;
  --radio-full: 9999px;

  /* — Sombras — */
  --sombra-sm: 0 2px 4px rgba(0, 0, 0, 0.05);
  --sombra-md: 0 4px 12px rgba(0, 0, 0, 0.08);
  --sombra-lg: 0 12px 30px rgba(0, 0, 0, 0.12);

  /* — Transiciones — */
  --transicion-rapida: 150ms ease;
  --transicion-base: 300ms ease;
  --transicion-lenta: 500ms ease;
}
```

## Decisiones de implementación, línea por línea:

- **:root:** Pseudo-clase que selecciona el elemento raíz de la página (el elemento `<html>`). Las variables definidas aquí se heredan y están disponibles **globalmente** en todo el documento.
- **Custom Properties (Variables CSS):** Se declaran usando el prefijo `--` y se recuperan mediante la función `var()`. Su gran ventaja es la centralización (cambias el valor en `:root` y se actualiza instantáneamente en todas las reglas) y la legibilidad visual (es más fácil entender qué es `--color-glaciar` que `#2e86ab`). Además, se pueden redefinir dinámicamente según el contexto (por ejemplo, en el modo oscuro).
- **Paleta de Colores Patagónicos:** Los nombres de colores elegidos evocan elementos de la provincia de Santa Cruz: glaciar (azul principal), lago (azul profundo para contrastes y hovers), atardecer (naranja para CTAs y acentos), estepa (dorado/ocre para aventura), nieve (off-white para fondos alternados) y cielo (celeste suave). Esto refuerza la identidad del portal.
- **--color-texto: #2b2b2b:** Se utiliza un gris piedra muy oscuro en lugar de negro puro (`#000`). Esto reduce la fatiga visual del usuario y proporciona un aspecto más elegante y pulido.
- **--radio-full: 9999px:** Un valor extremadamente alto de `border-radius` crea bordes completamente redondeados en forma de “píldora”. Funciona porque el radio de redondeado real se limita a la mitad del lado más corto del elemento, y `9999px` siempre excede ese valor.
- **--sombra-sm, --sombra-md, --sombra-lg:** Sombras difusas de color negro con opacidades del 5 %, 8 % y 12 % respectivamente. El uso de valores de opacidad muy bajos evita que las sombras luzcan sucias y pesadas, logrando un efecto de elevación y profundidad muy moderno.

## Paso 2.2: Reset y estilos base

```
*, ::before, ::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: var(--font-cuerpo);
  background-color: var(--color-fondo);
  color: var(--color-texto);
  line-height: 1.6;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  overflow-x: hidden;
}

a {
  text-decoration: none;
  color: inherit;
  transition: color var(--transicion-rapida);
}

ul {
  list-style: none;
}

img {
  display: block;
  max-width: 100%;
  height: auto;
}
```

## Decisiones de implementación, línea por línea:

- **Reset CSS:** Los distintos navegadores (Chrome, Firefox, Safari) aplican márgenes, rellenos y estilos tipográficos por defecto inconsistentes. El reset elimina estas variaciones para que nuestro diseño se comporte de forma idéntica en cualquier plataforma.
- **box-sizing: border-box:** Por defecto (content-box), la anchura (width) de un elemento no incluye el padding ni el borde. Al aplicar border-box, el padding y el borde se restan del ancho declarado, haciendo los cálculos de maquetación 100 % intuitivos.
  - *Ejemplo práctico:* Un div con width: 100 %, padding: 20px y border: 1px:
    - Sin border-box: El div real mide 100 % + 40px + 2px (desbordando la pantalla horizontalmente).
    - Con border-box: El div real mide exactamente 100 %, ajustando el relleno hacia adentro.
- **Selector universal \*, \*::before, \*::after:** El asterisco selecciona todos los elementos HTML, y las pseudo-classes de contenido generado aseguran que la propiedad box-sizing se aplique de forma consistente en cada rincón del DOM.
- **scroll-behavior: smooth:** Activa el desplazamiento suave nativo en el navegador al hacer clic en enlaces de anclaje (por ejemplo, al navegar de #inicio a #destinos), mejorando notablemente la experiencia de navegación del usuario.
- **body { display: flex; flex-direction: column; min-height: 100vh; }:** Esta combinación es la base del **Sticky Footer** (pie de página fijo abajo). El body se comporta como una caja flexible vertical que mide como mínimo el 100 % de la altura de la ventana visible (100vh). Esto nos permitirá empujar el footer hacia abajo usando un margen automático si hay poco contenido.
- **overflow-x: hidden:** Oculta cualquier desborde horizontal accidental provocado por elementos mal maquetados en móviles, evitando la aparición de barras de scroll horizontal molestas.
- **img { display: block; max-width: 100%; height: auto; }:** Por defecto, las imágenes son elementos de línea (inline), lo que genera un espacio vacío no deseado debajo de ellas (reservado para las colas de letras como la 'p' o 'g'). display: block elimina ese espacio. max-width: 100% e height: auto previenen que imágenes de alta resolución se desborden de sus contenedores padres y mantengan su relación de aspecto original de forma elástica.

### Paso 2.3: Clases comunes de layout

```
.contenedor {
  width: 90%;
  max-width: 1200px;
  margin-left: auto;
  margin-right: auto;
}

.seccion {
  padding: 40px 0;
}

.seccion-titulo {
  font-family: var(--font-titulo);
  font-size: 2rem;
  color: var(--color-lago);
  text-align: center;
  margin-bottom: 8px;
}

.seccion-subtitulo {
  font-size: 1rem;
  color: var(--color-texto-secundario);
  text-align: center;
  margin-bottom: 32px;
  max-width: 600px;
  margin-left: auto;
  margin-right: auto;
}

.oculto {
  display: none !important;
}

.visualmente-oculto {
  position: absolute;
  width: 1px;
}
```



```

height: 1px;
padding: 0;
margin: -1px;
overflow: hidden;
clip: rect(0, 0, 0, 0);
border: 0;
}

```

### Decisiones de implementación, línea por línea:

- **.contenedor:** Patrón de diseño “contenedor de ancho máximo centrado”. `width: 90%` proporciona un margen de seguridad del 5 % a cada lado en dispositivos móviles. `max-width: 1200px` evita que la línea de lectura se estire demasiado en monitores gigantesco (lo que dificulta el recorrido del ojo). `margin-left/right: auto` distribuye los márgenes laterales equitativamente, centrando el contenedor.
- **.seccion-subtitulo { max-width: 600px; }:** Limitar el ancho del subtítulo a 600px evita que las líneas de texto se extiendan demasiado, mejorando la legibilidad.
- **.oculto { display: none !important; }:** Oculta el elemento por completo. No se dibuja ni ocupa espacio en la distribución espacial. La directiva `!important` garantiza que esta regla prevalezca sobre cualquier otra regla de visualización. Es ideal para que JavaScript controle estados de la interfaz (como mostrar u ocultar la tarjeta de “Sin resultados”).
- **.visualmente-oculto: Diferencia crucial con display: none:**
  - `display: none` remueve el elemento del flujo visual y del árbol de accesibilidad (los lectores de pantalla lo ignoran).
  - `.visualmente-oculto` esconde el elemento físicamente del ojo humano (`width/height: 1px, clip: rect(...)`) pero lo preserva en el DOM. **Los lectores de pantalla sí lo leen.**
  - *Cuándo usarlo:* Etiquetas `<label>` de inputs que ya tienen placeholders legibles, textos de ayuda para botones que solo contienen un emoji o ícono, o textos descriptivos no visuales.

### Paso 2.4: Header y navegación (Mobile First)

```

.header {
  background-color: var(--color-superficie);
  border-bottom: 1px solid var(--color-borde);
  position: sticky;
  top: 0;
  z-index: 100;
  transition: background-color var(--transicion-base), box-shadow var(--transicion-base);
}

.header-contenido {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 0;
}

.logo {
  display: flex;
  align-items: center;
  gap: 8px;
  font-family: var(--font-titulo);
  font-size: 1.3rem;
  font-weight: 700;
  color: var(--color-lago);
}

.logo strong {
  color: var(--color-glaciador);
}

.menu-hamburguesa {

```

```

display: flex;
flex-direction: column;
justify-content: space-between;
width: 24px;
height: 18px;
background: transparent;
border: none;
cursor: pointer;
z-index: 200;
}

.hamburguesa-linea {
width: 100%;
height: 2px;
background-color: var(--color-lago);
border-radius: var(--radio-sm);
transition: transform var(--transicion-rapida), opacity var(--transicion-rapida);
}

.nav-principal {
position: absolute;
top: 100%;
left: 0;
width: 100%;
background-color: var(--color-superficie);
border-bottom: 3px solid var(--color-glaciador);
box-shadow: var(--sombra-md);
display: none;
z-index: 150;
}

.nav-principal.visible {
display: block;
animation: deslizarAbajo 300ms ease-out;
}

@keyframes deslizarAbajo {
from { transform: translateY(-10px); opacity: 0; }
to { transform: translateY(0); opacity: 1; }
}

.nav-lista {
display: flex;
flex-direction: column;
padding: 16px 0;
gap: 8px;
text-align: center;
}

.nav-enlace {
font-weight: 600;
font-size: 1.1rem;
color: var(--color-texto);
padding: 10px 20px;
transition: color var(--transicion-rapida), background-color var(--transicion-rapida);
}

.nav-enlace:hover {
color: var(--color-glaciador);
background-color: var(--color-nieve);
}

```

```

}

.nav-enlace.activo {
  color: var(--color-glaciar);
  background-color: var(--color-nieve);
  border-left: 4px solid var(--color-glaciar);
}

```

### Decisiones de implementación, línea por línea:

- **position: sticky; top: 0:** Fija el header en la parte superior del navegador durante el scroll. A diferencia de position: fixed (que se saca por completo de la maquetación física y requiere un margen superior compensatorio en el body), sticky respeta el flujo de maquetación natural y se comporta como fijo solo al rebasar el valor de top.
- **z-index e índices de apilamiento:**
  - z-index: 100 en el header asegura que el encabezado flote sobre las imágenes y tarjetas de contenido del scroll.
  - z-index: 150 en .nav-principal posiciona el menú desplegable por encima del cuerpo del sitio.
  - z-index: 200 en el botón .menu-hamburguesa garantiza que este permanezca clickeable e interactivo por encima del menú desplegable móvil.
- **.menu-hamburguesa y distribución de spans:** El ícono hamburguesa se maqueta utilizando Flexbox vertical (flex-direction: column y justify-content: space-between) separando equitativamente los 3 spans horizontales para formar la estructura de líneas.
- **.nav-principal { position: absolute; top: 100%; left: 0; display: none; }:** Por defecto en diseño Mobile First, el menú principal de navegación se oculta (display: none) y se posiciona absolutamente justo en el borde inferior del header (top: 100%).
- **.nav-principal.visible:** Cuando JavaScript inyecta la clase .visible, el menú se despliega y se ejecuta la animación @keyframes deslizarAbajo que aplica un fade-in (opacity de 0 a 1) y una sutil traslación vertical (translateY(-10px) a 0), suavizando el comportamiento de la UI.
- **.nav-enlace.activo:** El enlace activo se resalta con un borde izquierdo de color azul glaciar (border-left: 4px) en móviles. Este indicador de estado cambiará a un borde inferior (border-bottom) en la hoja de estilos responsiva de pantallas más anchas.

### Paso 2.5: Hero Banner

```

.hero {
  position: relative;
  background-image: url('../img/hero-santa-cruz.jpg');
  background-size: cover;
  background-position: center;
  min-height: 70vh;
  display: flex;
  align-items: center;
}

.hero-overlay {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: linear-gradient(to bottom, rgba(27, 27, 47, 0.3) 0%, rgba(27, 27, 47, 0.75) 100%);
  display: flex;
  align-items: center;
}

.hero-contenido {
  text-align: center;
  max-width: 800px;
  margin-left: auto;
  margin-right: auto;
  z-index: 10;
}

```

```
padding: 24px;
color: var(--color-nieve);
}

.hero-titulo {
font-family: var(--font-titulo);
font-size: clamp(2rem, 6vw, 3.5rem);
font-weight: 700;
line-height: 1.2;
margin-bottom: 16px;
text-shadow: 0 2px 8px rgba(0, 0, 0, 0.6);
}

.hero-subtitulo {
font-size: clamp(0.95rem, 2.5vw, 1.2rem);
font-weight: 300;
margin-bottom: 32px;
max-width: 650px;
margin-left: auto;
margin-right: auto;
text-shadow: 0 1px 4px rgba(0, 0, 0, 0.6);
}
```

#### Decisiones de implementación, línea por línea:

- **background-size: cover + background-position: center:** Instruye al navegador para escalar y recortar de forma inteligente la imagen de fondo de manera que cubra el 100 % de la sección Hero sin deformarse, manteniendo el centro de la imagen siempre visible.
- **min-height: 70vh:** Define un alto responsivo elástico. Mide como mínimo el 70 % del alto total del navegador. Se escala dinámicamente según el dispositivo sin necesidad de usar media queries de altura.
- **linear-gradient en el overlay:** Crea una máscara translúcida de color oscuro que va del 30 % al 75 % de opacidad. Esto genera un alto contraste con las letras blancas y garantiza que el texto del banner principal sea perfectamente legible sobre la imagen de fondo.
- **font-size: clamp(2rem, 6vw, 3.5rem): Tipografía fluida:**
  - clamp(mínimo, preferido, máximo) calcula el tamaño de fuente dinámicamente.
  - En móviles pequeños no bajará de 2rem.
  - En pantallas medianas se adaptará de forma fluida al 6vw (6 % del ancho de la ventana del navegador).
  - En monitores de escritorio grandes se limitará a 3.5rem.
  - *Ventaja:* Texto completamente fluido y adaptativo con una sola línea de código CSS, eliminando múltiples media queries redundantes.
- **text-shadow: 0 2px 8px rgba(0, 0, 0, 0.6):** Añade una sombra de texto difusa para separar visualmente las letras del fondo gráfico y garantizar la legibilidad en caso de imágenes de fondo complejas.

## 5. Tarea 3 — Sección Destinos: tarjetas con imágenes y badges

### Objetivo

Crear la sección de Destinos Turísticos con 8 tarjetas que incluyen imagen, badge de categoría, título, descripción y botón. Usar atributos data-\* para almacenar información de filtrado.

#### Paso 3.1: HTML de la sección Destinos

```
<section class="seccion destinos" id="destinos">
  <div class="contenedor">
    <h2 class="seccion-titulo">Destinos Turísticos</h2>
    <p class="seccion-subtitulo">
      Explorá los rincones más increíbles de Santa Cruz.
```

```

    Filtrá por categoría o buscá tu próximo destino.
  </p>

  <div class="destinos-grilla" id="destinos-grilla">
    <!-- Destino 1: Glaciar Perito Moreno -->
    <article
      class="tarjeta-destino"
      data-categoria="naturaleza"
      data-nombre="glaciar perito moreno"
    >
      <div class="tarjeta-destino-imagen">
        
        <span class="tarjeta-destino-badge">Naturaleza</span>
      </div>
      <div class="tarjeta-destino-info">
        <h3 class="tarjeta-destino-nombre">Glaciar Perito Moreno</h3>
        <p class="tarjeta-destino-descripcion">
          Uno de los pocos glaciares en el mundo que sigue avanzando.
          Sus 5 km de frente y 60 metros de altura sobre el agua
          lo convierten en un espectáculo natural único.
        </p>
        <a href="destino-detalle.html" class="btn btn-secundario">Ver más</a>
      </div>
    </article>

    <!-- Repetir estructura para los otros 7 destinos... -->
  </div>
</div>
</section>

```

### Decisiones de implementación, línea por línea:

- **data-categoria="naturaleza" y data-nombre="glaciar perito moreno"**: Atributos personalizados data-\* que nos permiten incrustar metadatos en elementos HTML. JavaScript los leerá de forma directa mediante la propiedad nativa `element.dataset` (por ejemplo, `tarjeta.dataset.categoria` devolverá "naturaleza"). Esto evita tener que parsear clases o IDs complejas.
- **data-nombre en minúsculas y sin acentos**: Almacenamos el nombre de búsqueda en minúsculas y sin tildes para simplificar sustancialmente la lógica de búsqueda en JavaScript, permitiendo búsquedas rápidas que ignoren tildes y mayúsculas.
- **loading="lazy"**: Activa el lazy loading (carga diferida) nativo. El navegador pospone la descarga de las imágenes que se encuentran fuera de la pantalla inicial hasta que el usuario hace scroll cerca de ellas. Esto acelera notablemente la carga inicial del sitio y ahorra datos móviles.
- **alt="..."**: Texto descriptivo alternativo para lectores de pantalla. Debe ser conciso pero lo suficientemente descriptivo del destino turístico para que un usuario con discapacidad visual lo comprenda con claridad. Si la imagen falla al cargar, este texto se renderiza en su lugar.
- **href="destino-detalle.html"**: Enlace semántico real que redirige a la página interna de detalles del destino correspondiente.

**Nota:** Los 8 destinos completos son: Glaciar Perito Moreno (naturaleza), Monte Fitz Roy (aventura), Cueva de las Manos (cultura), Bosque Petrificado (naturaleza), Lago Argentino (naturaleza), Puerto Deseado (aventura), Río Pinturas (cultura), Estancias Patagónicas (gastronomía).

### Paso 3.2: CSS de las tarjetas de destino

```
.destinos-grilla {
  display: grid;
  grid-template-columns: 1fr;
  gap: 32px;
}

.tarjeta-destino {
  background-color: var(--color-superficie);
  border-radius: var(--radio-lg);
  overflow: hidden;
  box-shadow: var(--sombra-md);
  display: flex;
  flex-direction: column;
  transition: transform var(--transicion-base), box-shadow var(--transicion-base);
  border: 1px solid var(--color-borde);
}

.tarjeta-destino:hover {
  transform: translateY(-5px);
  box-shadow: var(--sombra-lg);
}

.tarjeta-destino-imagen {
  position: relative;
  height: 200px;
  overflow: hidden;
}

.tarjeta-destino-imagen img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: transform var(--transicion-lenta);
}

.tarjeta-destino:hover .tarjeta-destino-imagen img {
  transform: scale(1.05);
}

.tarjeta-destino-badge {
  position: absolute;
  top: 12px;
  left: 12px;
  background-color: var(--color-lago);
  color: var(--color-nieve);
  padding: 4px 12px;
  border-radius: var(--radio-full);
  font-size: 0.8rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  z-index: 10;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
}

/* Colores de badges dinámicos por categoría (vía data-atributos) */
.tarjeta-destino[data-categoria="naturaleza"] .tarjeta-destino-badge {
  background-color: var(--color-glaciar);
}
```

```

.tarjeta-destino[data-categoria="aventura"] .tarjeta-destino-badge {
  background-color: var(--color-estepa);
}

.tarjeta-destino[data-categoria="cultura"] .tarjeta-destino-badge {
  background-color: var(--color-rosa);
}

.tarjeta-destino[data-categoria="gastronomia"] .tarjeta-destino-badge {
  background-color: var(--color-atardecer);
}

.tarjeta-destino-info {
  padding: 24px;
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}

.tarjeta-destino-nombre {
  font-family: var(--font-titulo);
  font-size: 1.35rem;
  color: var(--color-lago);
  margin-bottom: 12px;
}

.tarjeta-destino-descripcion {
  font-size: 0.95rem;
  color: var(--color-texto-secundario);
  margin-bottom: 20px;
  line-height: 1.5;
  flex-grow: 1;
}

.tarjeta-destino-info .btn {
  align-self: flex-start;
}

```

### Decisiones de implementación, línea por línea:

- **display: grid; grid-template-columns: 1fr;** Estructura el contenedor en una grilla de una única columna en pantallas de dispositivos móviles, ocupando todo el ancho. El espaciado de gap: 32px separa los artículos de forma clara y limpia.
- **display: flex; flex-direction: column; en .tarjeta-destino:** Cada tarjeta es un flexbox vertical. Esto permite organizar el contenido (imagen superior y bloque de información inferior) de manera secuencial y fluida.
- **overflow: hidden en .tarjeta-destino-imagen:** Escondemos cualquier fragmento gráfico que sobrepase los 200px de altura de la caja de imagen. Esto es crucial para lograr el efecto zoom al pasar el mouse por encima de la tarjeta sin que desborde el contenedor.
- **object-fit: cover:** Indica al navegador que escale y rellene la caja de la imagen preservando la relación de aspecto original de la fotografía, recortando las proporciones sobrantes de manera simétrica (similar a background-size: cover en fondos CSS).
- **position: absolute en el badge:** Permite colocar de forma precisa la etiqueta de categoría sobre la esquina superior izquierda de la imagen (top/left: 12px). El contenedor padre .tarjeta-destino-imagen tiene position: relative para servir de anclaje de posición. z-index: 10 asegura que no se oculte tras la foto.
- **.tarjeta-destino[data-categoria="..."]:** CSS permite seleccionar elementos en función de sus atributos. Esto nos permite asociar un color de fondo diferente a los badges basándonos en la categoría (data-categoria) de la tarjeta padre de forma centralizada y sin necesidad de escribir múltiples clases modificadoras redundantes en HTML.
- **flex-grow: 1 en .tarjeta-destino-info y en .tarjeta-destino-descripcion:**
  - flex-grow: 1 en el contenedor de información estira esta caja para que ocupe todo el espacio sobrante de la tarjeta.
  - Al aplicarse también a la descripción, esta caja de texto se estira empujando al botón “Ver más” de forma automática hacia el borde inferior.

- **Ventaja:** Si una tarjeta tiene una descripción de 3 líneas y otra de 5 líneas, los botones “Ver más” permanecerán alineados exactamente en el mismo nivel horizontal en todas las tarjetas de la fila.
- **align-self: flex-start en el botón:** Por defecto, un contenedor flexible en columna estiraría el botón al 100 % de la anchura lateral. flex-start restringe el tamaño del botón a la anchura natural de su propio contenido de texto.
- **transform: scale(1.05) en hover:** Al pasar el mouse por encima del artículo, la imagen se agranda suavemente un 5 % en un tiempo de var(--transicion-lenta), creando una micro-animación de zoom inmersiva que invita al clic.

## 6. Tarea 4 — Secciones Testimonios y Newsletter

### Objetivo

Crear las secciones de Testimonios de viajeros (prueba social) y el formulario de suscripción al Newsletter.

#### Paso 4.1: HTML de Testimonios

```
<section class="seccion testimonios" id="testimonios">
  <div class="contenedor">
    <h2 class="seccion-titulo">Lo que dicen los viajeros</h2>
    <p class="seccion-subtitulo">
      Experiencias reales de quienes ya descubrieron la magia de Santa Cruz.
    </p>

    <div class="testimonios-grilla">
      <article class="tarjeta-testimonio">
        <blockquote class="testimonio-texto">
          "Ver el desprendimiento del Perito Moreno fue la experiencia más impactante de mi vida..."
        </blockquote>
        <div class="testimonio-autor">
          <span class="testimonio-avator" aria-hidden="true">👤</span>
          <div>
            <cite class="testimonio-nombre">María González</cite>
            <span class="testimonio-origen">Buenos Aires, Argentina</span>
          </div>
        </div>
      </article>

      <!-- Repetir para otros 2 testimonios... -->
    </div>
  </div>
</section>
```

#### Decisiones de implementación, línea por línea:

- **<blockquote>**: Representa una sección de texto que es una cita de otra fuente (en este caso, un testimonio). Es la etiqueta semántica correcta para maquetar opiniones y citas de usuarios.
- **<cite>**: Representa el autor o la fuente original de la cita enlazada. Por defecto se muestra en cursiva en la mayoría de navegadores.
- **Cada testimonio es un <article>**: Se encapsula en un artículo porque cada opinión es una unidad de contenido independiente, con significado completo por sí misma.
- **Prueba Social (Social Proof)**: Los testimonios reales de viajeros funcionan como prueba social, un principio psicológico de persuasión según el cual las personas confían más en un destino o servicio al ver que otros ya lo han disfrutado positivamente.

#### Paso 4.2: CSS de Testimonios

```
.testimonios {
  background-color: var(--color-nieve);
}
```



```

.testimonios-grilla {
  display: grid;
  grid-template-columns: 1fr;
  gap: 24px;
}

.tarjeta-testimonio {
  background-color: var(--color-superficie);
  padding: 32px 24px 24px;
  border-radius: var(--radio-lg);
  position: relative;
  box-shadow: var(--sombra-sm);
  display: flex;
  flex-direction: column;
  gap: 20px;
  border: 1px solid var(--color-borde);
}

.testimonio-texto {
  font-style: italic;
  color: var(--color-texto);
  font-size: 0.95rem;
  position: relative;
  line-height: 1.6;
}

.testimonio-texto::before {
  content: '"';
  font-size: 4.5rem;
  font-family: var(--font-titulo);
  color: var(--color-glaciador);
  opacity: 0.15;
  position: absolute;
  top: -32px;
  left: -8px;
  line-height: 1;
}

.testimonio-autor {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: auto;
}

.testimonio-avatar {
  font-size: 2rem;
  background-color: var(--color-nieve);
  width: 48px;
  height: 48px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--radio-full);
  flex-shrink: 0;
}

.testimonio-nombre {
  display: block;
  font-weight: 700;
}

```

```

color: var(--color-lago);
font-size: 0.95rem;
font-style: normal;
}

.testimonio-origen {
display: block;
font-size: 0.8rem;
color: var(--color-texto-secundario);
}

```

### Decisiones de implementación, línea por línea:

- **.testimonio-texto::before**: Pseudo-elemento decorativo que inserta un carácter de comilla doble gigante antes del texto. Al configurarlo con `position: absolute`, `font-size: 4.5rem` y una opacidad baja del 15 % (`opacity: 0.15`), flota sutilmente al fondo detrás de las primeras letras, logrando un estilo tipográfico premium sin alterar el código HTML con caracteres decorativos vacíos.
- **margin-top: auto en .testimonio-autor**: Como la tarjeta de testimonio es una caja flexible en columna, aplicar margen superior automático al bloque del autor empuja esta fila firmemente hacia la base de la tarjeta, logrando una alineación perfecta aunque unos testimonios sean más largos que otros.
- **font-style: normal en .testimonio-nombre**: Como el texto de la cita ya está formateado en cursiva (`font-style: italic`), forzamos que el nombre dentro de la etiqueta `<cite>` se muestre normal, anulando la cursiva por defecto del navegador para mejorar el contraste de lectura.

### Paso 4.3: HTML de Newsletter

```

<section class="seccion newsletter" id="newsletter">
  <div class="contenedor newsletter-contenido">
    <h2 class="seccion-titulo">¿Querés recibir novedades?</h2>
    <p class="seccion-subtitulo">
      Suscribite a nuestro boletín y recibí las mejores recomendaciones para tu viaje a Santa Cruz.
    </p>

    <form class="newsletter-form" id="form-newsletter">
      <label for="email-newsletter" class="visually-oculto">Tu dirección de email</label>
      <input
        type="email"
        id="email-newsletter"
        class="newsletter-input"
        placeholder="Tu email, ej: nombre@ejemplo.com"
        required
        autocomplete="email"
      >
      <button type="submit" class="btn btn-primario newsletter-btn">Suscribirme</button>
    </form>

    <p class="newsletter-mensaje" id="newsletter-mensaje" role="alert" aria-live="polite"></p>
  </div>
</section>

```

### Decisiones de implementación, línea por línea:

- **<label for="email-newsletter" class="visually-oculto">**: Aunque visualmente no queremos mostrar una etiqueta textual porque el placeholder ya cumple esa función de guía para usuarios videntes, los lectores de pantalla la requieren obligatoriamente (los placeholders no siempre se anuncian). La clase `.visually-oculto` la esconde visualmente sin removerla del árbol de accesibilidad (cosa que `display: none` sí haría).
- **type="email"**: Valida nativamente que el texto ingresado tenga un formato válido de dirección de correo electrónico (que contenga @ y un dominio formal), mostrando advertencias antes del envío.
- **required**: Atributo de validación nativa que impide el envío del formulario si el campo de texto se encuentra completamente

vacío.

- **autocomplete="email"**: Facilita la experiencia de usuario (UX) sugiriendo direcciones de correo que el usuario haya guardado en el navegador, acelerando la suscripción.
- **role="alert" + aria-live="polite"**: Atributos de ARIA para notificaciones dinámicas. Cuando JavaScript inserte el mensaje de confirmación en #newsletter-mensaje, el sintetizador de voz lo anunciará de forma automática en cuanto el lector esté libre, sin interrumpir abruptamente la lectura actual.

#### Paso 4.4: CSS de Newsletter

```
.newsletter {
  background-color: var(--color-lago);
  color: var(--color-nieve);
  padding: 48px 0;
  text-align: center;
}

.newsletter .seccion-titulo {
  color: var(--color-nieve);
}

.newsletter .seccion-subtitulo {
  color: var(--color-cielo);
  margin-bottom: 24px;
}

.newsletter-form {
  display: flex;
  flex-direction: column;
  gap: 12px;
  margin-top: 24px;
}

.newsletter-input {
  padding: 14px 20px;
  border-radius: var(--radio-full);
  border: 2px solid transparent;
  font-size: 1rem;
  width: 100%;
  outline: none;
  background-color: rgba(255, 255, 255, 0.95);
  font-family: inherit;
  transition: background-color var(--transicion-rapida), border-color var(--transicion-rapida), box-shadow var(--transicion-rapida);
}

.newsletter-input:focus {
  background-color: #ffffff;
  border-color: var(--color-atardecer);
  box-shadow: 0 0 0 3px rgba(241, 143, 1, 0.3);
}

.newsletter-btn {
  width: 100%;
  font-size: 1rem;
  padding: 14px 28px;
}

.newsletter-mensaje {
  margin-top: 16px;
  font-weight: 600;
}
```

```
font-size: 0.95rem;
}

.newsletter-mensaje.exito {
  color: #2ecc71;
}

.newsletter-mensaje.error {
  color: #e74c3c;
}
```

#### Decisiones de implementación, línea por línea:

- **Fondo de Contraste (--color-lago):** El cambio brusco a un color de fondo oscuro y llamativo rompe la inercia del scroll e indica visualmente al usuario que está ante una sección interactiva importante de conversión.
- **outline: none y Focus Accesible en el input:** El navegador dibuja por defecto un anillo de enfoque azul de bajo contraste que quitamos con outline: none. Sin embargo, es **obligatorio por accesibilidad** restaurar esta visualización para usuarios que navegan con teclado (Tabulador). Lo logramos modificando el color de borde a naranja (--color-atardecer) y rodeándolo de un brillo translúcido con box-shadow.
- **Clases .exito y .error:** Estilos dinámicos inyectados por JavaScript. Se asocian a convenciones de colores universales (verde #2ecc71 para indicar acierto y rojo #e74c3c para indicar errores) para que la respuesta de estado sea comprensible al instante.

## 7. Tarea 5 — Footer y Formulario de Contacto

### Objetivo

Crear el Footer con 4 columnas (descripción, enlaces, contacto, redes sociales) y la sección de Contacto con formulario de consulta + información de oficina.

#### Paso 5.1: HTML del Footer

```
<footer class="footer" id="footer">
  <div class="contenedor footer-contenido">
    <div class="footer-columna">
      <h3 class="footer-titulo">Descubrí Santa Cruz</h3>
      <p class="footer-descripcion">
        Portal de turismo de la provincia de Santa Cruz, Patagonia Argentina.
        Información oficial sobre destinos, actividades y servicios turísticos.
      </p>
    </div>

    <div class="footer-columna">
      <h3 class="footer-titulo">Enlaces</h3>
      <nav aria-label="Navegación del footer">
        <ul class="footer-enlaces">
          <li><a href="index.html">Inicio</a></li>
          <li><a href="#destinos">Destinos</a></li>
          <li><a href="#contacto">Contacto</a></li>
          <li><a href="#testimonios">Testimonios</a></li>
        </ul>
      </nav>
    </div>

    <div class="footer-columna">
      <h3 class="footer-titulo">Contacto</h3>
      <address class="footer-contacto">
        <p>📍 Av. Presidente Kirchner 863, Río Gallegos</p>
        <p>☎ (02966) 42-2702</p>
      </address>
    </div>
  </div>
</footer>
```

```

        <p>
            ✉
            <a href="mailto:turismo@santacruz.gob.ar">turismo@santacruz.gob.ar</a>
        </p>
    </address>
</div>

<div class="footer-columna">
    <h3 class="footer-titulo">Seguinos</h3>
    <div class="footer-redes">
        <a href="#" class="red-social" aria-label="Facebook" title="Facebook">📘</a>
        <a href="#" class="red-social" aria-label="Instagram" title="Instagram">📷</a>
        <a href="#" class="red-social" aria-label="Twitter / X" title="Twitter / X">🐦</a>
        <a href="#" class="red-social" aria-label="YouTube" title="YouTube">▶</a>
    </div>
</div>
</div>

<div class="footer-inferior">
    <div class="contenedor">
        <p>
            © 2026 Descubrí Santa Cruz – Proyecto académico para Laboratorio Web 1.
            Todos los derechos reservados.
        </p>
    </div>
</div>
</footer>

```

### Decisiones de implementación, línea por línea:

- **<footer>**: Etiqueta semántica de HTML5 que define el pie de página de la aplicación. Suele albergar datos de propiedad intelectual, enlaces rápidos de navegación secundaria, información de contacto y enlaces a redes sociales.
- **<address>**: Etiqueta semántica destinada específicamente a contener datos de contacto físicos o digitales de personas u organizaciones autoras del documento. Rellena el bloque en cursiva por defecto en los navegadores.
- **aria-label en enlaces de redes sociales**: Dado que las redes sociales contienen únicamente un emoji en su interior (📘, 📷, etc.), los lectores de pantalla los verbalizarían de forma confusa. Usamos `aria-label="Facebook"` para que los sintetizadores de voz enuncien de forma directa “Facebook, enlace”.
- **title en los enlaces**: Proporciona un pequeño mensaje emergente visual (tooltip) que se muestra al usuario vidente al posicionar el cursor sobre el ícono.
- **Estructura en 4 columnas**: El footer se maquetó en el CSS mediante una grilla responsiva que en pantallas móviles se apila verticalmente en una sola columna y se divide en 4 columnas en monitores grandes.

### Paso 5.2: CSS del Footer

```

.footer {
    background-color: #1b1b2f;
    color: var(--color-nieve);
    padding: 48px 0 0;
    font-size: 0.9rem;
    margin-top: auto;
}

.footer-contenido {
    display: grid;
    grid-template-columns: 1fr;
    gap: 32px;
    margin-bottom: 32px;
}

```

```

.footer-titulo {
  font-family: var(--font-titulo);
  font-size: 1.25rem;
  color: var(--color-glaciar);
  position: relative;
  padding-bottom: 8px;
}

/* Línea decorativa debajo del título del footer */
.footer-titulo::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  width: 40px;
  height: 2px;
  background-color: var(--color-atardecer);
}

.footer-descripcion {
  color: #a0a0b8;
  line-height: 1.6;
}

.footer-enlaces {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.footer-enlaces a {
  color: #a0a0b8;
  transition: color var(--transicion-rapida), padding-left var(--transicion-rapida);
}

.footer-enlaces a:hover {
  color: var(--color-atardecer);
  padding-left: 6px;
}

.red-social {
  font-size: 1.4rem;
  background-color: rgba(255, 255, 255, 0.05);
  width: 40px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--radio-full);
  transition: background-color var(--transicion-rapida), transform var(--transicion-rapida);
}

.red-social:hover {
  background-color: var(--color-glaciar);
  transform: translateY(-3px);
}

.footer-inferior {
  border-top: 1px solid rgba(255, 255, 255, 0.06);
  padding: 20px 0;
}

```

```

text-align: center;
color: #7f7f9c;
font-size: 0.8rem;
}

```

### Decisiones de implementación, línea por línea:

- **margin-top: auto (Sticky Footer):** Junto con el comportamiento flexible en column del body, empuja de forma automática la caja del footer hasta el borde inferior de la pantalla. Esto evita que el pie de página flote a mitad de la ventana si la página tiene poco contenido textual.
- **Colores fijos en lugar de variables:** El footer usa valores estáticos (#1b1b2f e #a0a0b8) en lugar de las variables redefinibles del :root. Esto garantiza que el pie de página mantenga siempre su estilo oscuro, independientemente de si el usuario cambia el sistema operativo a modo claro u oscuro.
- **.footer-titulo::after:** Pseudo-elemento con position: absolute y bottom: 0 que dibuja una línea horizontal naranja de decoración justo debajo de los títulos de las columnas.
- **padding-left: 6px en hover:** Aplica una animación suave al pasar el mouse por encima de los enlaces rápidos, creando una sensación de deslizamiento horizontal interactiva muy moderna.

### Paso 5.3: HTML del Formulario de Contacto

```

<section class="seccion contacto" id="contacto">
  <div class="contenedor grid-contacto">
    <!-- Columna Izquierda: Formulario de consulta -->
    <article class="contacto-formulario-caja">
      <h2 class="contacto-seccion-titulo">Envía tu consulta</h2>
      <p class="contacto-seccion-subtitulo">Todos los campos marcados con (*) son obligatorios.</p>

      <form class="form-contacto" id="form-contacto" novalidate>
        <!-- Campo 1: Nombre completo -->
        <div class="campo-grupo">
          <label for="nombre" class="campo-label">Nombre completo *</label>
          <input
            type="text"
            id="nombre"
            name="nombre"
            class="campo-input"
            placeholder="Ingresa tu nombre y apellido"
            required
            autocomplete="name"
            aria-describedby="nombre-error"
            aria-invalid="false"
          >
          <span id="nombre-error" class="error-mensaje" role="alert" aria-live="polite"></span>
        </div>

        <!-- Campo 2: Correo electrónico -->
        <div class="campo-grupo">
          <label for="email" class="campo-label">Correo electrónico *</label>
          <input
            type="email"
            id="email"
            name="email"
            class="campo-input"
            placeholder="ejemplo@correo.com"
            required
            autocomplete="email"
            aria-describedby="email-error"
            aria-invalid="false"
          >

```

```

    <span id="email-error" class="error-mensaje" role="alert" aria-live="polite"></span>
</div>

<!-- Campo 3: Teléfono (opcional) -->
<div class="campo-grupo">
    <label for="telefono" class="campo-label">Teléfono (opcional)</label>
    <input
        type="tel"
        id="telefono"
        name="telefono"
        class="campo-input"
        placeholder="Ej: +54 9 2966 123456"
        autocomplete="tel"
        pattern="[+]?[0-9\s\-\-]{7,20}"
        aria-describedby="telefono-error"
        aria-invalid="false"
    >
    <span id="telefono-error" class="error-mensaje" role="alert" aria-live="polite"></span>
</div>

<!-- Campo 4: Motivo de consulta -->
<div class="campo-grupo">
    <label for="motivo" class="campo-label">Motivo de consulta *</label>
    <select id="motivo" name="motivo" class="campo-select" required aria-describedby="motivo-error"
        aria-invalid="false">
        <option value="">Seleccioná un motivo...</option>
        <option value="info-general">Información general</option>
        <option value="senderos">Senderos y Trekking</option>
        <option value="alojamiento">Alojamiento y Servicios</option>
        <option value="clima">Estado de rutas y Clima</option>
        <option value="otro">Otro motivo</option>
    </select>
    <span id="motivo-error" class="error-mensaje" role="alert" aria-live="polite"></span>
</div>

<!-- Campo 5: Mensaje -->
<div class="campo-grupo">
    <label for="mensaje" class="campo-label">Mensaje *</label>
    <textarea
        id="mensaje"
        name="mensaje"
        class="campo-textarea"
        rows="5"
        placeholder="Escribí tu consulta en detalle (mínimo 20 caracteres)..."
        required
        minlength="20"
        aria-describedby="mensaje-error"
        aria-invalid="false"
    ></textarea>
    <span id="mensaje-error" class="error-mensaje" role="alert" aria-live="polite"></span>
</div>

<!-- Campo 6: Checkbox boletín de novedades -->
<div class="campo-grupo-checkbox">
    <input type="checkbox" id="newsletter-contacto" name="newsletter" class="campo-checkbox">
    <label for="newsletter-contacto" class="campo-label-checkbox">
        Deseo recibir novedades y eventos turísticos de Santa Cruz
    </label>
</div>

```



```

<!-- Botón de envío -->
<button type="submit" class="btn btn-primario form-btn">Enviar consulta</button>

<!-- Mensaje de éxito/error general -->
<p id="contacto-mensaje-general" class="contacto-mensaje-general" role="alert" aria-live="polite"></p>
</form>
</article>

<!-- Columna Derecha: Información de la oficina oficial -->
<article class="contacto-info-caja">
  <h2 class="contacto-seccion-titulo">Oficina de Informes Oficial</h2>
  <p class="contacto-seccion-subtitulo">Secretaría de Estado de Turismo de Santa Cruz.</p>

  <address class="contacto-direccion-bloque">
    <div class="contacto-item">
      <span class="contacto-icone" aria-hidden="true">📍</span>
      <div>
        <strong>Dirección física:</strong>
        <p>Av. Presidente Kirchner 863, Río Gallegos, Santa Cruz (CP 9400)</p>
      </div>
    </div>

    <div class="contacto-item">
      <span class="contacto-icone" aria-hidden="true">☎️</span>
      <div>
        <strong>Teléfono de contacto:</strong>
        <p><a href="tel:+542966422702">(02966) 42-2702</a></p>
      </div>
    </div>

    <div class="contacto-item">
      <span class="contacto-icone" aria-hidden="true">✉️</span>
      <div>
        <strong>Correo institucional:</strong>
        <p><a href="mailto:turismo@santacruz.gob.ar">turismo@santacruz.gob.ar</a></p>
      </div>
    </div>

    <div class="contacto-item">
      <span class="contacto-icone" aria-hidden="true">🕒</span>
      <div>
        <strong>Horario de atención:</strong>
        <p>Lunes a Viernes de 08:00 a 20:00 hs. Sábados y Domingos de 10:00 a 18:00 hs.</p>
      </div>
    </div>
  </address>

  <!-- Mapa placeholder (simulación visual) -->
  <div class="mapa-placeholder">
    <div class="mapa-contenido">
      <span class="mapa-icone" aria-hidden="true">🗺️</span>
      <strong>Mapa de ubicación</strong>
      <p>Av. Pres. Kirchner 863, Río Gallegos</p>
    </div>
  </div>
</article>
</div>
</section>

```

Decisiones de implementación, línea por línea:

- **novalidate en <form>**: Desactiva las burbujas y alertas de validación nativas por defecto del navegador web. Esto es necesario porque dichos mensajes nativos son extremadamente difíciles de estilizar y su comportamiento varía de forma inconsistente entre navegadores. Al usar `novalidate`, tomamos el control total en JavaScript para implementar una validación accesible y homogénea.
- **aria-describedby="[id]-error"**: Vincula de forma semántica el campo del input con la caja de texto que contendrá el mensaje de error de validación. Al navegar con teclado o lector de pantalla sobre el input, el dispositivo verbalizará de forma automática el error asociado, además de la etiqueta normal.
- **aria-invalid="true/false"**: Atributo de estado ARIA dinámico. Indica a las tecnologías de asistencia si el campo ingresado por el usuario tiene fallos (`true`) o está correcto (`false`).
- **role="alert" + aria-live="polite" en error-mensaje**: Permite que el lector de pantalla enuncie inmediatamente el texto del mensaje de error tan pronto como JavaScript modifique el contenido del span.

#### Paso 5.4: CSS del Formulario de Contacto

```
.grid-contacto {
  display: grid;
  grid-template-columns: 1fr;
  gap: 48px;
}

.contacto-seccion-titulo {
  font-family: var(--font-titulo);
  font-size: 1.5rem;
  color: var(--color-lago);
  margin-bottom: 8px;
}

.form-contacto {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.campo-grupo {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.campo-label {
  font-weight: 600;
  font-size: 0.95rem;
  color: var(--color-texto);
}

.campo-input,
.campo-select,
.campo-textarea {
  width: 100%;
  padding: 12px 16px;
  border-radius: var(--radio-md);
  border: 1px solid var(--color-borde);
  background-color: var(--color-superficie);
  color: var(--color-texto);
  font-family: inherit;
  font-size: 0.95rem;
  outline: none;
  transition: border-color var(--transicion-rapida), box-shadow var(--transicion-rapida);
}
```

```

.campo-input:focus,
.campo-select:focus,
.campo-textarea:focus {
  border-color: var(--color-glaciado);
  box-shadow: 0 0 3px rgba(46, 134, 171, 0.2);
}

/* Estilo para inputs inválidos (validación dinámica con JS) */
.campo-input[aria-invalid="true"],
.campo-select[aria-invalid="true"],
.campo-textarea[aria-invalid="true"] {
  border-color: #e74c3c !important;
  box-shadow: 0 0 3px rgba(231, 76, 60, 0.15) !important;
}

.error-mensaje {
  color: #e74c3c;
  font-size: 0.8rem;
  font-weight: 600;
  margin-top: 4px;
  min-height: 18px;
  display: block;
}

.campo-grupo-checkbox {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  margin-top: 8px;
}

.campo-checkbox {
  width: 18px;
  height: 18px;
  margin-top: 2px;
  cursor: pointer;
}

.campo-label-checkbox {
  font-size: 0.9rem;
  color: var(--color-texto-secundario);
  cursor: pointer;
  user-select: none;
}

.contacto-mensaje-general {
  font-weight: 600;
  font-size: 1rem;
  margin-top: 8px;
  text-align: center;
}

.contacto-mensaje-general.exito {
  color: #2ecc71;
}

.contacto-mensaje-general.error {
  color: #e74c3c;
}

```

```

.contacto-direccion-bloque {
  display: flex;
  flex-direction: column;
  gap: 20px;
  font-style: normal;
}

.contacto-item {
  display: flex;
  gap: 16px;
  align-items: flex-start;
}

.contacto-item .contacto-icone {
  font-size: 1.5rem;
  background-color: var(--color-nieve);
  width: 44px;
  height: 44px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--radio-md);
  flex-shrink: 0;
}

.contacto-item strong {
  display: block;
  font-size: 0.95rem;
  color: var(--color-lago);
}

.contacto-item p {
  font-size: 0.9rem;
  color: var(--color-texto-secundario);
}

.mapa-placeholder {
  background-color: var(--color-nieve);
  border: 1px solid var(--color-borde);
  border-radius: var(--radio-lg);
  padding: 40px 24px;
  text-align: center;
  margin-top: 32px;
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 200px;
  position: relative;
  overflow: hidden;
  box-shadow: var(--sombra-sm);
}

.mapa-placeholder::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: linear-gradient(135deg, rgba(46, 134, 171, 0.05) 0%, rgba(22, 78, 99, 0.05) 100%);
}

```

```

    z-index: 0;
  }

  .mapa-contenido {
    position: relative;
    z-index: 1;
  }

  .mapa-icone {
    font-size: 2.5rem;
    display: block;
    margin-bottom: 8px;
  }

  .mapa-placeholder strong {
    display: block;
    font-family: var(--font-titulo);
    color: var(--color-lago);
    margin-bottom: 4px;
  }

  .mapa-placeholder p {
    font-size: 0.85rem;
    color: var(--color-texto-secundario);
  }

```

#### Decisiones de implementación, línea por línea:

- **Selector [aria-invalid="true"]:** En lugar de depender de añadir y remover una clase CSS de error en JavaScript, usamos selectores de atributos en CSS. Esto garantiza la total sincronización de la presentación de error (borde rojo) con el estado real de accesibilidad leído por tecnologías de asistencia.
- **min-height: 18px en .error-mensaje:** Reserva espacio de altura física en la pantalla para contener el mensaje de error. Si el span está vacío, este no colapsa a 0px. Esto previene molestos saltos en el layout visual cuando aparece o desaparece una alerta de validación.
- **flex-shrink: 0 en .contacto-icone:** Dado que el bloque de contacto utiliza flexbox y el texto explicativo puede ser extenso en algunas direcciones, forzamos al ícono a no contraerse (shrink a 0) para asegurar que mantenga su medida fija e intacta de 44x44px.
- **::before en .mapa-placeholder:** Inyecta un gradiente estético de fondo de simulación cartográfica. En producción, esta caja contendría un mapa dinámico cargado vía <iframe>.

## 8. Tarea 6 — JavaScript: Menú hamburguesa móvil

### Objetivo

Agregar interactividad al menú hamburguesa: abrir/cerrar al hacer clic, actualizar atributos ARIA, y cerrar automáticamente al navegar.

#### Paso 6.1: Referencias al DOM y función toggle

```

const btnMenu = document.getElementById('btn-menu');
const navPrincipal = document.getElementById('nav-principal');

function toggleMenu() {
  const estaAbierto = navPrincipal.classList.contains('visible');

  if (estaAbierto) {
    navPrincipal.classList.remove('visible');
    btnMenu.classList.remove('activo');
  }
}

```

```

    btnMenu.setAttribute('aria-expanded', 'false');
    btnMenu.setAttribute('aria-label', 'Abrir menú de navegación');
  } else {
    navPrincipal.classList.add('visible');
    btnMenu.classList.add('activo');
    btnMenu.setAttribute('aria-expanded', 'true');
    btnMenu.setAttribute('aria-label', 'Cerrar menú de navegación');
  }
}

if (btnMenu && navPrincipal) {
  btnMenu.addEventListener('click', toggleMenu);
}

```

#### Decisiones de implementación, línea por línea:

- **DOM (Document Object Model):** Representación en forma de árbol de objetos estructurados del documento HTML que el navegador genera en memoria. Cada etiqueta HTML se traduce en un objeto o “nodo” interactivo que JavaScript puede leer, modificar, crear o eliminar de forma dinámica.
- **document.getElementById('btn-menu'):** Busca en el árbol del DOM el elemento que posea el atributo id indicado y devuelve su referencia de objeto. Es el método de consulta **más rápido** y eficiente que ofrece el navegador, ya que los IDs deben ser únicos.
- **navPrincipal.classList.contains('visible'):** Comprueba si la lista de clases del elemento contiene la clase CSS indicada. Devuelve true o false. Es mucho más limpio y moderno que realizar búsquedas de subcadenas con `className.indexOf()`.
- **classList.add() / classList.remove():** Añade o quita clases CSS de la lista de clases del elemento de forma controlada sin afectar ni alterar el resto de las clases existentes en la etiqueta HTML (a diferencia de reescribir con `element.className = ...`).
- **setAttribute('aria-expanded', 'true/false'):** Modifica dinámicamente el valor de los atributos ARIA en el HTML para indicar el estado de colapsado/desplegado del menú. Esto garantiza que las personas ciegas que navegan con lectores de pantalla conozcan el estado visual real del menú interactivo en tiempo real.
- **Máquina de estados en toggleMenu():** Implementa un flujo lógico condicional de control. Si el menú está abierto, lo cierra y actualiza ARIA; si está cerrado, lo abre y actualiza ARIA. A esto se le conoce como máquina de estados lógica básica.
- **addEventListener('click', toggleMenu):** Registra el método listener para que se ejecute cuando ocurra el evento de clic del usuario sobre el botón. Evitamos usar atributos HTML antiguos como `onclick="..."` para mantener la separación de responsabilidades: el marcado estructural queda en HTML y la conducta en JavaScript.
- **Validación defensiva if (btnMenu && navPrincipal):** Comprueba que las referencias de objeto devueltas no sean nulas antes de intentar añadir un event listener. Si en el futuro cambiamos el diseño de la página HTML y se remueve el menú de navegación, esta condición defensiva evitará que el script se interrumpa abruptamente lanzando el clásico error de consola: "Cannot read property of null".

#### Paso 6.2: Función para cerrar el menú

```

function cerrarMenuMovil() {
  if (navPrincipal && navPrincipal.classList.contains('visible')) {
    navPrincipal.classList.remove('visible');
    btnMenu.classList.remove('activo');
    btnMenu.setAttribute('aria-expanded', 'false');
    btnMenu.setAttribute('aria-label', 'Abrir menú de navegación');
  }
}

```

#### Decisiones de implementación, línea por línea:

- **Principio DRY (Don't Repeat Yourself):** Agrupamos todo el bloque de código encargado de cerrar el menú desplegable móvil en una función modular independiente y reutilizable. En lugar de duplicar este mismo conjunto de líneas en varios bloques de eventos (al hacer clic en enlaces de navegación, al cambiar el hash de la URL, etc.), invocamos simplemente a `cerrarMenuMovil()`. Esto hace el código fácil de mantener.

- **Guarda defensiva adicional:** Validamos que `navPrincipal` sea una referencia válida y que contenga la clase `.visible` antes de ejecutar la remoción y los cambios de atributos de forma segura.

## 9. Tarea 7 — JavaScript: Filtrado y búsqueda de destinos

### Objetivo

Implementar el filtrado por categoría y la búsqueda por nombre de los destinos turísticos, con actualización instantánea del contador y mensaje “sin resultados”.

#### Paso 7.1: Variables de estado

```
const botonesFiltro = document.querySelectorAll('.filtro-btn');
const buscadorDestino = document.getElementById('buscador-destino');
const tarjetasDestino = document.querySelectorAll('.tarjeta-destino');
const sinResultados = document.getElementById('sin-resultados');
const cantidadResultados = document.getElementById('cantidad-resultados');

let categoriaActiva = 'todos';
let busquedaTexto = '';
```

#### Decisiones de implementación, línea por línea:

- `document.querySelectorAll('.filtro-btn')`: Busca TODOS los elementos que coinciden con un selector CSS. Devuelve una `NodeList` (similar a un array) que podemos recorrer con `forEach`. A diferencia de un array nativo, no tiene todos sus métodos (como `map` o `filter` en navegadores antiguos), pero sí soporta `forEach` para iteración simple.
- `let categoriaActiva = 'todos'`: Variable de **estado**. Guarda qué categoría está seleccionada actualmente. Es la “única fuente de verdad” para saber qué mostrar.
- `let busquedaTexto = ''`: Otra variable de estado. Guarda el texto que el usuario escribió en el buscador.
- **¿Por qué variables de estado? (Patrón Estado + Renderizado / State + Render):**
  - En el desarrollo web profesional, se separa el **estado** (los datos puros que describen la aplicación en memoria, como `categoriaActiva` y `busquedaTexto`) del **renderizado** o dibujo en pantalla (la representación visual del DOM).
  - La variable de estado actúa como la “Única Fuente de Verdad” (**Single Source of Truth**). Esto significa que no le preguntamos al DOM qué botón está pintado o qué hay en la caja de texto cada vez que filtramos; en cambio, consultamos directamente las variables en memoria.
  - Esto habilita un **flujo de datos unidireccional**: la interacción del usuario modifica las variables de estado, y el cambio de estado dispara la función `filtrarDestinos()` que redibuja o actualiza la UI. Así, desacoplamos totalmente la interacción física de la lógica de pintado del DOM, haciendo el código mucho más fácil de mantener y escalar.

#### Paso 7.2: HTML de los filtros (agregar antes de la grilla de destinos)

```
<div class="filtros-categorias" id="filtros-categorias" role="group" aria-label="Filtrar por categoría">
  <button type="button" class="filtro-btn activo" data-filtro="todos" aria-pressed="true">🌐 Todos</button>
  <button type="button" class="filtro-btn" data-filtro="naturaleza" aria-pressed="false">🌿 Naturaleza</button>
  <button type="button" class="filtro-btn" data-filtro="aventura" aria-pressed="false">🏔️ Aventura</button>
  <button type="button" class="filtro-btn" data-filtro="cultura" aria-pressed="false">🎭 Cultura</button>
  <button type="button" class="filtro-btn" data-filtro="gastronomia" aria-pressed="false">🍷 Gastronomía</button>
</div>

<div class="filtros-buscador">
  <label for="buscador-destino" class="visualmente-oculto">Buscar destino por nombre</label>
  <input type="search" id="buscador-destino" class="buscador-input"
    placeholder="🔍 Buscar destino por nombre..." autocomplete="off">
</div>

<p class="filtros-contador" id="filtros-contador" aria-live="polite">
  Mostrando <span id="cantidad-resultados">8</span> de 8 destinos
```

```

</p>

<p class="sin-resultados oculto" id="sin-resultados" role="status">
  🙄 No encontramos destinos con esa búsqueda.
</p>

```

### Decisiones de implementación, línea por línea:

- **role="group" + aria-label="Filtrar por categoría"**: Indica que estos botones forman un grupo relacionado. Los lectores de pantalla anuncian “grupo, Filtrar por categoría”.
- **aria-pressed**: Indica si un botón está presionado (true) o no (false). Es como un interruptor.
- **data-filtro="naturaleza"**: Atributo personalizado que indica qué categoría filtra cada botón. JavaScript lo lee con `boton.dataset.filtro`.
- **type="search"**: Similar a `type="text"` pero el navegador puede agregar un botón de “limpiar” (×) automáticamente.
- **aria-live="polite" en el contador**: Le dice al lector de pantalla “anunciá este cambio cuando estés libre”, sin interrumpir al usuario.
- **.oculto en sin-resultados**: Por defecto está oculto. JavaScript le quitará esta clase cuando no haya resultados.

### Paso 7.3: CSS de los filtros

```

.filtros-categorias {
  display: flex;
  gap: 8px;
  overflow-x: auto;
  padding-bottom: 8px;
  scrollbar-width: none;
}

.filtros-categorias::-webkit-scrollbar {
  display: none;
}

.filtro-btn {
  flex-shrink: 0;
  background-color: var(--color-superficie);
  border: 1px solid var(--color-borde);
  color: var(--color-texto);
  padding: 8px 16px;
  border-radius: var(--radio-full);
  cursor: pointer;
  font-weight: 600;
  font-family: inherit;
  font-size: 0.9rem;
  transition: all var(--transicion-rapida);
}

.filtro-btn:hover {
  background-color: var(--color-nieve);
  border-color: var(--color-glaciar);
  color: var(--color-glaciar);
  transform: translateY(-1px);
}

.filtro-btn.activo {
  background-color: var(--color-glaciar);
  border-color: var(--color-glaciar);
  color: var(--color-nieve);
  box-shadow: 0 2px 6px rgba(46, 134, 171, 0.3);
}

```



```

.buscador-input {
  width: 100%;
  padding: 12px 20px;
  border-radius: var(--radio-full);
  border: 1px solid var(--color-borde);
  font-size: 1rem;
  outline: none;
  background-color: var(--color-superficie);
  font-family: inherit;
  transition: border-color var(--transicion-rapida), box-shadow var(--transicion-rapida);
}

.buscador-input:focus {
  border-color: var(--color-glaciar);
  box-shadow: 0 0 3px rgba(46, 134, 171, 0.2);
}

.sin-resultados {
  text-align: center;
  font-size: 1.15rem;
  padding: 48px 20px;
  color: var(--color-texto-secundario);
}

```

#### Decisiones de implementación, línea por línea:

- **overflow-x: auto:** Permite el desplazamiento horizontal de los botones de filtro si no caben en el ancho de la pantalla (especialmente en dispositivos móviles con pantallas angostas). Evita tener que apilarlos verticalmente, lo cual ocuparía demasiado espacio en la pantalla de un celular.
- **-webkit-overflow-scrolling: touch:** Aunque no está explícitamente listada en este bloque básico, es una buena recomendación para navegadores iOS antiguos para darles un scroll con inercia nativa más suave al tacto.
- **scrollbar-width: none + ::-webkit-scrollbar { display: none }:** Patrón de maquetación moderno que oculta visualmente la barra de desplazamiento horizontal (tanto en navegadores basados en Webkit/Blink como Chrome, Safari y Edge, como en Firefox), manteniendo intacta la funcionalidad táctil de scroll. Esto da un aspecto de “carrusel de botones” muy limpio y estético en móviles.
- **flex-shrink: 0:** Elemento clave. Evita que los botones hijos de un contenedor Flexbox se encojan e intenten forzarse a caber dentro de los límites visuales de la pantalla. Al ponerlo en 0, cada botón mantiene su ancho intrínseco (calculado según su padding y texto), obligando al contenedor a desbordar horizontalmente y a habilitar el scroll de forma natural.
- **.filtro-btn.activo:** Cambia el fondo a azul glaciar, el color de borde y el color de texto a blanco. JavaScript alternará esta clase cuando el usuario haga clic. También agrega una sombra suave (box-shadow) para darle relieve tridimensional al botón seleccionado.

#### Paso 7.4: Función de filtrado

```

function filtrarDestinos() {
  let contadorVisibles = 0;

  tarjetasDestino.forEach(tarjeta => {
    const categoriaTarjeta = tarjeta.dataset.categoria;
    const nombreTarjeta = tarjeta.dataset.nombre;

    const coincideCategoria = (categoriaActiva === 'todos' || categoriaTarjeta === categoriaActiva);
    const coincideTexto = (busquedaTexto === '' || nombreTarjeta.includes(busquedaTexto));

    if (coincideCategoria && coincideTexto) {
      tarjeta.classList.remove('oculto');
      contadorVisibles++;
    } else {
      tarjeta.classList.add('oculto');
    }
  });
}

```

```

    }
  });

  if (cantidadResultados) {
    cantidadResultados.textContent = contadorVisibles;
  }

  if (contadorVisibles === 0) {
    sinResultados.classList.remove('oculto');
  } else {
    sinResultados.classList.add('oculto');
  }
}

```

### Decisiones de implementación, línea por línea:

- **tarjeta.dataset.categoria / tarjeta.dataset.nombre:** Leen los atributos personalizados data-categoria y data-nombre que definimos en el HTML de cada destino. En JavaScript, el acceso a atributos data-\* se realiza de manera nativa y directa a través del objeto dataset, convirtiendo el nombre en camelCase (por ejemplo, data-mi-variable se lee como dataset.miVariable).
- **coincideCategoria:** Lógica de disyunción (operador **OR** lógico ||). Evalúa a true si la variable de estado categoriaActiva es 'todos' (lo cual significa que no hay filtro activo) **O** si el atributo data-categoria de la tarjeta es exactamente igual a la categoriaActiva. Esto permite que al seleccionar “Todos”, pasen todas las tarjetas sin importar su categoría individual.
- **nombreTarjeta.includes(búsquedaTexto):** El método includes() verifica si un string contiene a otro. Devuelve true si el texto de búsqueda está incluido en cualquier parte del nombre del destino (por ejemplo, si buscamos "gla" en "glaciar perito moreno"). Si búsquedaTexto está vacío "", la condición búsquedaTexto === '' evalúa a true de inmediato, evitando hacer una búsqueda de texto.
- **coincideCategoria && coincideTexto:** Lógica de conjunción (operador **AND** lógico &&). El destino cumple con los requisitos y se muestra **únicamente si ambas condiciones son verdaderas a la vez**. Esto permite cruzar ambos filtros: por ejemplo, buscar “glaciar” pero sólo dentro de la categoría “naturaleza”.
- **tarjeta.classList.add('oculto') / remove('oculto'):**
  - Para ocultar o mostrar los elementos, agregamos o quitamos la clase .oculto (definida en el CSS Reset con display: none !important).
  - **Ventaja de rendimiento:** Alternar clases CSS es infinitamente más eficiente que crear y destruir elementos en el DOM con document.createElement() o .innerHTML. Al quitar o poner la clase, el navegador sólo realiza un reflow/repaint visual sin tener que reconstruir la jerarquía del árbol de nodos de la página o perder los controladores de eventos ya registrados en las tarjetas.
- **cantidadResultados.textContent = contadorVisibles:** Escribe el número total de destinos que pasaron el filtro combinado en el elemento <span> correspondiente, actualizando al usuario en tiempo real sobre cuántos elementos se están visualizando.
- **Mensaje sin resultados (sinResultados.classList):** Si la variable acumuladora contadorVisibles termina en 0, significa que ningún destino superó los filtros cruzados. En ese caso, removemos la clase .oculto del elemento #sin-resultados para mostrar un aviso amigable en pantalla; de lo contrario, lo ocultamos. Como este elemento tiene role="status" en el HTML, el navegador informará de manera accesible este estado al usuario.

### Paso 7.5: Eventos de filtrado

```

botonesFiltro.forEach(boton => {
  boton.addEventListener('click', () => {
    botonesFiltro.forEach(b => {
      b.classList.remove('activo');
      b.setAttribute('aria-pressed', 'false');
    });

    boton.classList.add('activo');
    boton.setAttribute('aria-pressed', 'true');

    categoriaActiva = boton.dataset.filtro;
    filtrarDestinos();
  });
});

```

```
});

if (buscadorDestino) {
  buscadorDestino.addEventListener('input', (e) => {
    let valorInput = e.target.value.trim().toLowerCase();

    valorInput = valorInput
      .replace(/[áâãä]/g, 'a')
      .replace(/[éêëè]/g, 'e')
      .replace(/[íîï]/g, 'i')
      .replace(/[óôõö]/g, 'o')
      .replace(/[úûüù]/g, 'u');

    busquedaTexto = valorInput;
    filtrarDestinos();
  });
}
```

Decisiones de implementación, línea por línea:

Decisiones de implementación, línea por línea:

- **botonesFiltro.forEach(boton => { ... })**: Registra un listener de clics individual para cada botón de categoría. Cuando el usuario hace clic:
  1. Recorremos nuevamente todos los botones (`botonesFiltro.forEach(b => { ... })`) para limpiar la clase visual `.activo` y restablecer el estado de accesibilidad `aria-pressed="false"`. Esto garantiza un comportamiento de tipo radio-button (sólo uno activo a la vez).
  2. Aplicamos la clase `.activo` y el atributo `aria-pressed="true"` exclusivamente al botón que recibió el clic.
  3. Modificamos la variable de estado `categoriaActiva` leyendo el valor del atributo personalizado `data-filtro` a través de `boton.dataset.filtro`.
  4. Llamamos a `filtrarDestinos()`, la cual leerá este nuevo estado en memoria y actualizará la visibilidad de los destinos en la UI.
- **El evento 'input' vs 'change' o 'keyup'**:
  - Usamos el evento `input` porque se dispara **inmediatamente** cada vez que el valor del campo cambia (al presionar una tecla, borrar con retroceso, pegar texto con el mouse, o limpiar el campo usando el botón nativo de búsqueda ✕).
  - A diferencia de `change` (que se dispara recién cuando el usuario quita el foco del `input`) o `keyup` (que no se activa si el usuario pega texto usando el menú contextual del botón derecho del mouse), `input` garantiza que la búsqueda sea fluida y en tiempo real.
- **e.target.value.trim().toLowerCase()**:
  - `e.target` hace referencia al elemento `<input>` que disparó el evento.
  - `.value` obtiene la cadena de texto actual ingresada por el usuario.
  - `.trim()` elimina cualquier espacio en blanco al inicio o al final del texto ingresado (evitando que una búsqueda accidental de "el chalten " falle por el espacio sobrante al final).
  - `.toLowerCase()` convierte toda la cadena a minúsculas, asegurando que la comparación sea insensible a mayúsculas/minúsculas (case-insensitive).
- **Normalización de acentos y diacríticos (Mapeo)**:
  - En español es muy común omitir las tildes al buscar. Si el usuario busca "rio" (sin acento) y el destino se llama "Río Pinturas" (con acento), la búsqueda fallaría por la diferencia entre la 'i' y la 'í'.
  - Para solucionar esto, aplicamos un encadenamiento de métodos `.replace()` utilizando **Expresiones Regulares (Regex)**.
- **replace(/[áâãä]/g, 'a')**:
  - Los corchetes `[...]` definen una **clase de caracteres**, lo que significa "coincidir con cualquiera de estas letras específicas".
  - La letra `g` al final de la expresión regular `/pattern/g` es la bandera **global**. Esto le indica a JavaScript que reemplace **todas** las ocurrencias de estas letras acentuadas encontradas en la cadena, no sólo la primera.
  - Al realizar esto para las cinco vocales en minúsculas, convertimos cadenas como "vía láctea" en "via lactea", permitiendo una búsqueda sumamente flexible y amigable para el estudiante y el usuario final.

## 10. Tarea 8 — JavaScript: Validación del formulario de contacto

### Objetivo

Validar el formulario de contacto en tiempo real, mostrar mensajes de error accesibles por campo, y dar feedback general de éxito/error.

#### Paso 8.1: Validación en el submit

```
const formContacto = document.getElementById('form-contacto');

if (formContacto) {
  const inputNombre = document.getElementById('nombre');
  const inputEmail = document.getElementById('email');
  const inputTelefono = document.getElementById('telefono');
  const selectMotivo = document.getElementById('motivo');
  const textareaMensaje = document.getElementById('mensaje');
  const mensajeGeneral = document.getElementById('contacto-mensaje-general');

  formContacto.addEventListener('submit', function(e) {
    e.preventDefault();
    let formularioValido = true;

    // 1. Validar Nombre
    const nombreError = document.getElementById('nombre-error');
    if (inputNombre.value.trim().length < 3) {
      nombreError.textContent = 'El nombre debe tener al menos 3 caracteres.';
      inputNombre.setAttribute('aria-invalid', 'true');
      formularioValido = false;
    } else {
      nombreError.textContent = '';
      inputNombre.setAttribute('aria-invalid', 'false');
    }

    // 2. Validar Email
    const emailError = document.getElementById('email-error');
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(inputEmail.value.trim())) {
      emailError.textContent = 'Ingresá una dirección de correo válida.';
      inputEmail.setAttribute('aria-invalid', 'true');
      formularioValido = false;
    } else {
      emailError.textContent = '';
      inputEmail.setAttribute('aria-invalid', 'false');
    }

    // 3. Validar Teléfono (opcional)
    const telefonoError = document.getElementById('telefono-error');
    const telValue = inputTelefono.value.trim();
    const telRegex = /^[+]?[0-9\s\-\]{7,20}$/;
    if (telValue !== '' && !telRegex.test(telValue)) {
      telefonoError.textContent = 'Formato inválido (mínimo 7 números).';
      inputTelefono.setAttribute('aria-invalid', 'true');
      formularioValido = false;
    } else {
      telefonoError.textContent = '';
      inputTelefono.setAttribute('aria-invalid', 'false');
    }

    // 4. Validar Motivo
    const motivoError = document.getElementById('motivo-error');
```

```

if (selectMotivo.value === '') {
  motivoError.textContent = 'Seleccioná un motivo para tu consulta.';
  selectMotivo.setAttribute('aria-invalid', 'true');
  formularioValido = false;
} else {
  motivoError.textContent = '';
  selectMotivo.setAttribute('aria-invalid', 'false');
}

// 5. Validar Mensaje
const mensajeError = document.getElementById('mensaje-error');
if (textareaMensaje.value.trim().length < 20) {
  mensajeError.textContent = 'El mensaje es demasiado corto (mínimo 20 caracteres).';
  textareaMensaje.setAttribute('aria-invalid', 'true');
  formularioValido = false;
} else {
  mensajeError.textContent = '';
  textareaMensaje.setAttribute('aria-invalid', 'false');
}

// 6. Mostrar resultado general
if (formularioValido) {
  mensajeGeneral.className = 'contacto-mensaje-general exito';
  mensajeGeneral.textContent = '¡Tu consulta fue enviada con éxito! Muy pronto nos pondremos en contacto.';
  formContacto.reset();
} else {
  mensajeGeneral.className = 'contacto-mensaje-general error';
  mensajeGeneral.textContent = 'Por favor, corregí los errores marcados antes de enviar.';
}
});

```

### Decisiones de implementación, línea por línea:

- **e.preventDefault():** Intercepta el evento de envío del formulario. Por defecto, cuando un elemento `<form>` se envía, el navegador intenta enviar los datos haciendo una petición HTTP POST o GET, lo cual recarga toda la página. Como este es un desarrollo de lado cliente (sin servidor o backend conectado), cancelar la recarga nativa con `preventDefault()` nos permite realizar validaciones con JavaScript y mostrar un cartel de éxito o error sin perder el estado o las variables actuales de la página.
- **let formularioValido = true:** Variable de control tipo **bandera (flag)**. Iniciamos el proceso asumiendo de manera optimista que el formulario es correcto. A medida que evaluamos cada campo, si alguno no supera su validación específica, cambiamos este flag a `false` y actualizamos sus respectivos mensajes de error visuales y atributos de accesibilidad. Al final del bloque, sólo enviaremos el formulario si la bandera se mantiene en `true`.
- **Expresión regular para validar Email (emailRegex):**
  - Las expresiones regulares (regex) son patrones de búsqueda complejos para validar formatos de texto.
  - `/` y `/:` Delimitan el inicio y fin de la expresión regular en JavaScript.
  - `^`: Fuerza a que la validación empiece exactamente al inicio de la cadena.
  - `[^\s@]+`: Clase de caracteres negada. El acento circunflejo `^` dentro de corchetes significa “cualquier carácter excepto los indicados”. En este caso, coincide con uno o más caracteres que **no** sean espacios (`\s`) ni el símbolo arroba (`@`).
  - `@`: Exige la presencia obligatoria de una arroba literal en esa posición.
  - `[^\s@]+`: Exige uno o más caracteres válidos después de la arroba que no contengan espacios ni otra arroba.
  - `\.`: Exige un punto literal. La barra invertida `\` sirve para “escapar” el punto, ya que en expresiones regulares el punto libre `.` significa “cualquier carácter”.
  - `[^\s@]+`: Exige uno o más caracteres que no sean espacio ni arroba para el dominio final (por ejemplo, `com`, `org`, `edu.ar`).
  - `$`: Fuerza a que el patrón termine exactamente al finalizar la cadena, evitando que el usuario meta texto inválido después del correo.
- **Expresión regular para validar Teléfono (telRegex):**
  - `/^[+]?[0-9\s\-\-]{7,20}$/`
  - `[+]?`: El símbolo más `+` es opcional (indicado por el signo de interrogación `?`). Esto sirve para prefijos internacionales (por ejemplo, `+54`).

- `[0-9\s\ - ]`: Permite números del 0 al 9, espacios en blanco (`\s`) o guiones medios (`\ -`). Nuevamente, el guion se escapa con barra invertida para que no se confunda con un rango (como `0-9`).
- `{7,20}`: Cuantificador. Exige que el conjunto anterior (números, espacios o guiones) tenga una longitud mínima de 7 caracteres y máxima de 20.
- `$` y `^`: Garantizan que toda la cadena completa cumpla estrictamente con esta estructura.
- **`telValue !== '' && !telRegex.test(telValue)`**: El teléfono es un campo **opcional** en el formulario de contacto. Si el usuario lo deja completamente vacío, no hay error. Pero si ingresa algún valor (`telValue !== ''`), entonces sí lo validamos con la expresión regular (`!telRegex.test(telValue)`) para asegurarnos de que no sea texto sin sentido o números incompletos.
- **`aria-invalid="true/false"`**: Sincroniza el estado del campo con lectores de pantalla. Cuando el campo falla la validación, le asignamos `true` para advertir visual y auditivamente a los usuarios con discapacidades. El navegador lee el error `span` correspondiente y asocia el `input` gracias al atributo `aria-describedby` que preparamos en el HTML (Tarea 5).
- **`mensajeGeneral.className = 'contacto-mensaje-general exito'`**:
  - `className` reemplaza **todas** las clases del elemento de una sola vez, a diferencia de `classList.add()` que sólo añade clases de forma selectiva.
  - Limpiar la propiedad y redefinirla como `'contacto-mensaje-general exito'` o `'contacto-mensaje-general error'` es una excelente manera de resetear clases dinámicas del elemento general y asegurarnos de que no se mezclen los estilos del cartel de éxito (verde) con el de error (rojo) si el usuario intenta enviar el formulario varias veces.
- **`formContacto.reset()`**: Método nativo del elemento formulario que limpia automáticamente todos sus campos (`inputs`, `textareas`, `selects`) restableciéndolos a sus valores por defecto tras un envío exitoso, listos para una nueva consulta.

## Paso 8.2: Validación en tiempo real

```
const campos = [inputNombre, inputEmail, inputTelefono, selectMotivo, textareaMensaje];
campos.forEach(function(campo) {
  campo.addEventListener('input', function() {
    if (campo.getAttribute('aria-invalid') === 'true') {
      campo.setAttribute('aria-invalid', 'false');
      const errorSpan = document.getElementById(campo.id + '-error');
      if (errorSpan) {
        errorSpan.textContent = '';
      }
    }
  });
});
```

### Decisiones de implementación, línea por línea:

- **`const campos = [inputNombre, inputEmail, inputTelefono, selectMotivo, textareaMensaje]`**: Creamos un array en memoria que agrupa todas las referencias a los elementos interactivos del formulario.
- **`campos.forEach(function(campo) { ... })`**: En lugar de copiar y pegar el mismo event listener para cada uno de los cinco campos individuales, usamos `forEach` para iterar sobre el array. Esto nos permite asociar la lógica a todos los campos en un único bloque de código limpio y compacto.
- **Validación en tiempo real (evento `'input'`)**: El evento `input` se dispara con cada pulsación de tecla o modificación del valor del campo. Escuchar este evento nos permite limpiar el estado de error de forma inmediata en cuanto el usuario empieza a corregir lo que escribió. Esto mejora enormemente la experiencia de usuario (UX), ya que el feedback visual negativo (borde rojo, mensaje de error) no permanece en pantalla una vez subsanado el problema.
- **`if (campo.getAttribute('aria-invalid') === 'true')`**: Condición optimizadora de accesibilidad y rendimiento. Sólo ejecutamos la limpieza si el campo estaba marcado previamente como inválido. De este modo, evitamos hacer operaciones innecesarias en el DOM (re-escribir atributos y vaciar el texto del `span`) en cada pulsación normal de tecla si el campo ya estaba correcto.
- **`campo.setAttribute('aria-invalid', 'false')`**: Restablece el estado de accesibilidad del campo indicando que su valor actual ya no es inválido.
- **`campo.id + '-error'`**: Generación dinámica del selector de error. Siguiendo el principio DRY, si nuestro `input` de nombre tiene el id "nombre", el `span` donde mostramos su error tiene el id "nombre-error". Al concatenar el id del elemento con la cadena `'-error'`, podemos buscar dinámicamente el `span` correspondiente en el DOM (`document.getElementById(campo.id + '-error')`) sin importar cuál de los cinco campos estemos procesando. Esto hace que el código sea altamente reutilizable y elegante.

## 11. Tarea 9 — JavaScript: Newsletter y navegación por hash

### Objetivo

Implementar la suscripción al newsletter con feedback visual, y la navegación por hash que permite usar los botones “Atrás/Adelante” del navegador.

#### Paso 9.1: Lógica del newsletter

```
const formNewsletter = document.getElementById('form-newsletter');
const emailNewsletter = document.getElementById('email-newsletter');
const newsletterMensaje = document.getElementById('newsletter-mensaje');

function mostrarMensajeNewsletter(mensaje, tipo) {
  newsletterMensaje.className = 'newsletter-mensaje';
  newsletterMensaje.classList.add(tipo);
  newsletterMensaje.textContent = mensaje;
}

function procesarSuscripcion(e) {
  e.preventDefault();
  const email = emailNewsletter.value.trim();

  if (email === '') {
    mostrarMensajeNewsletter('Por favor, ingresá un correo electrónico.', 'error');
    return;
  }

  mostrarMensajeNewsletter('¡Gracias por suscribirte! Muy pronto vas a recibir novedades.', 'exito');
  formNewsletter.reset();
}

if (formNewsletter) {
  formNewsletter.addEventListener('submit', procesarSuscripcion);
}
```

#### Decisiones de implementación, línea por línea:

- **newsletterMensaje.className = 'newsletter-mensaje':** Reseteamos las clases CSS del elemento del mensaje a su estado inicial. Al hacer esto de manera explícita antes de agregar el tipo con `classList.add(tipo)`, prevenimos que se acumulen las clases de estados anteriores (como que el elemento tenga la clase `.exito` y `.error` al mismo tiempo tras múltiples intentos de suscripción).
- **textContent frente a innerHTML (Seguridad contra XSS):**
  - **Concepto clave (XSS - Cross-Site Scripting):** Si utilizáramos `innerHTML` para insertar textos dinámicos generados a partir de lo que escribe un usuario, un atacante podría inyectar código HTML malicioso o scripts ejecutables (por ejemplo, `<img src=invalid onerror="alert('hack')">` o `<script>...</script>`). El navegador interpretaría ese código e interrumpiría la seguridad del sitio.
  - Al utilizar `.textContent`, el navegador trata cualquier cadena ingresada estrictamente como **texto plano literal**. Si el texto contiene caracteres como `<` o `>`, los renderiza visualmente en pantalla de manera segura pero no los interpreta como nodos de código.
  - **Buenas prácticas:** Siempre que no necesites renderizar etiquetas HTML explícitas, prefiere `.textContent` por sobre `.innerHTML`.
- **Accesibilidad (role="alert" + aria-live="polite" en el HTML):**
  - El elemento `#newsletter-mensaje` está pre-configurado en el HTML con el rol semántico `role="alert"` y el atributo `aria-live="polite"`.
  - Cuando JavaScript altera dinámicamente el `.textContent` de este nodo vacío, los lectores de pantalla capturan ese cambio de texto en tiempo real y lo anuncian de forma audible al usuario sin necesidad de que éste tenga que mover el foco de

navegación hasta el mensaje de error. Esto es fundamental para una interfaz accesible e inclusiva.

## Paso 9.2: Navegación por hash

```
const enlacesNav = document.querySelectorAll('.nav-enlace');

function actualizarEnlaceActivo() {
  const hashActual = window.location.hash || '#inicio';

  enlacesNav.forEach(enlace => {
    const href = enlace.getAttribute('href');
    let hashEnlace;

    if (href.startsWith('#')) {
      hashEnlace = href;
    } else if (href === 'index.html' || href === '/') {
      hashEnlace = '#inicio';
    } else {
      return;
    }

    if (hashEnlace === hashActual) {
      enlace.classList.add('activo');
      enlace.setAttribute('aria-current', 'page');
    } else {
      enlace.classList.remove('activo');
      enlace.removeAttribute('aria-current');
    }
  });
}

function navegarAHash(hash) {
  if (!hash || hash === '#inicio') {
    window.scrollTo({ top: 0, behavior: 'smooth' });
    return;
  }
  const seccionDestino = document.querySelector(hash);
  if (seccionDestino) {
    seccionDestino.scrollIntoView({ behavior: 'smooth' });
  }
}

function manejarClickEnlace(e) {
  const enlace = e.currentTarget;
  const href = enlace.getAttribute('href');

  if (href.startsWith('#') || href === 'index.html') {
    e.preventDefault();
    let hash = (href === 'index.html') ? '#inicio' : href;

    history.pushState(null, '', hash);
    actualizarEnlaceActivo();
    navegarAHash(hash);
    cerrarMenuMovil();
  }
}

enlacesNav.forEach(enlace => {
  enlace.addEventListener('click', manejarClickEnlace);
});
```



```

window.addEventListener('hashchange', function() {
  actualizarEnlaceActivo();
  navegarAHash(window.location.hash);
});

document.addEventListener('DOMContentLoaded', function() {
  const hashInicial = window.location.hash;
  if (hashInicial) {
    setTimeout(function() {
      navegarAHash(hashInicial);
    }, 100);
  }
  actualizarEnlaceActivo();
});

```

### Decisiones de implementación, línea por línea:

- **Concepto de Client-Side Routing (Ruteo en el Cliente):** En las aplicaciones de página única modernas (SPAs), no recargamos la página completa para ir a otra sección. En su lugar, interceptamos la navegación y usamos la URL (a través del #hash) para cambiar lo que se ve en pantalla. Esto da una experiencia de usuario instantánea y fluida.
- **window.location.hash:** Propiedad de lectura/escritura del navegador que contiene el fragmento actual de la URL que empieza con # (por ejemplo, #destinos o #contacto). Si estamos en la página inicial sin anclaje, devuelve una cadena vacía "". En nuestro código, usamos window.location.hash || '#inicio' para establecer un valor por defecto seguro.
- **history.pushState(null, '', hash):**
  - Modifica la barra de direcciones del navegador en tiempo real, agregando el hash (ej. index.html#contacto), **pero sin disparar una recarga de página ni emitir el evento hashchange de forma nativa.**
  - **Diferencia técnica:** Si hiciéramos window.location.hash = hash, el navegador realizaría un scroll abrupto e instantáneo al elemento con ese ID. Al usar pushState(), silenciamos ese comportamiento nativo para poder controlarlo nosotros de manera refinada con JavaScript, actualizando manualmente las clases de los botones de navegación y aplicando nuestro scroll personalizado animado.
- **e.preventDefault() en enlaces:** Cancela la acción por defecto del enlace <a> (que saltaría de golpe al anclaje y recargaría si apunta a otra ruta). Al detener esto, tomamos el control absoluto de la transición de navegación mediante JavaScript.
- **scrollIntoView({ behavior: 'smooth' }) y window.scrollTo():**
  - seccionDestino.scrollIntoView({ behavior: 'smooth' }) es una API nativa moderna del navegador que desplaza de forma animada (“suave”) la pantalla hasta posicionar la sección seleccionada en la vista del usuario.
  - Para volver al principio de todo (cuando el hash es #inicio), no hay una sección específica con ese ID, por lo que usamos window.scrollTo({ top: 0, behavior: 'smooth' }), que desplaza el scroll general de la ventana al punto inicial superior (0, 0).
- **Escuchar el evento 'hashchange':**
  - Si el usuario navega haciendo clic en los links del menú, nuestro controlador de eventos maneja todo. ¿Pero qué pasa si el usuario presiona los botones de **Atrás** o **Adelante** de su navegador?
  - El navegador cambia el hash de la URL, pero no hace clic en ningún botón. Para soportar esto y que la interfaz siga respondiendo correctamente, escuchamos el evento hashchange en el objeto global window. Cada vez que el hash varíe externamente, redibujamos el enlace activo en el menú y navegamos a la sección correspondiente.
- **DOMContentLoaded y el retraso defensivo (setTimeout(..., 100)):**
  - El evento DOMContentLoaded se dispara tan pronto como el HTML está completamente descargado y analizado por el navegador, permitiéndonos ejecutar código JS de forma segura.
  - Sin embargo, cuando entramos a la página directamente con un hash (por ejemplo, alguien comparte el link midominio.com/#contacto), el navegador necesita una fracción de segundo para cargar el CSS, renderizar las fuentes tipográficas y calcular las dimensiones reales y posiciones de cada sección.
  - Al meter navegarAHash(hashInicial) dentro de un setTimeout(..., 100), retrasamos la animación 100 milisegundos. Esta pequeña pausa defensiva le da tiempo al motor de renderizado del navegador para terminar de armar el diseño visual, garantizando que el scroll termine exactamente en la posición vertical correcta de la sección.

## 12. Tarea 10 — Responsive design y modo oscuro

### Objetivo

Agregar los media queries para adaptar el layout a tablet y desktop, y el soporte automático para modo oscuro del sistema.

### Paso 10.1: Media Queries (Tablet y Desktop)

```
/* — Tablet (min-width: 768px) — */
@media (min-width: 768px) {
  .seccion { padding: 60px 0; }

  .menu-hamburguesa { display: none; }

  .nav-principal {
    position: static;
    display: block;
    width: auto;
    background-color: transparent;
    border-bottom: none;
    box-shadow: none;
  }

  .nav-lista {
    flex-direction: row;
    padding: 0;
    gap: 8px;
  }

  .nav-enlace.activo {
    border-left: none;
    border-bottom: 3px solid var(--color-glaciar);
    border-radius: var(--radio-md) var(--radio-md) 0 0;
  }

  .tarjetas-grilla { grid-template-columns: repeat(2, 1fr); }
  .destinos-grilla { grid-template-columns: repeat(2, 1fr); gap: 24px; }
  .testimonios-grilla { grid-template-columns: repeat(2, 1fr); }

  .newsletter-form { flex-direction: row; gap: 8px; }
  .newsletter-input { flex-grow: 1; }
  .newsletter-btn { width: auto; }

  .footer-contenido { grid-template-columns: repeat(2, 1fr); }
  .grid-contacto { grid-template-columns: 1fr 1fr; gap: 40px; }
}

/* — Desktop (min-width: 1024px) — */
@media (min-width: 1024px) {
  .seccion { padding: 80px 0; }

  .tarjetas-grilla { grid-template-columns: repeat(4, 1fr); }
  .destinos-grilla { grid-template-columns: repeat(3, 1fr); gap: 32px; }
  .testimonios-grilla { grid-template-columns: repeat(3, 1fr); }
  .footer-contenido { grid-template-columns: repeat(4, 1fr); gap: 24px; }
}
```

Decisiones de implementación, línea por línea:

- **Estrategia Mobile First (El móvil primero):**
  - Comenzamos a codificar los estilos base (fuera de cualquier media query) para las pantallas más pequeñas (celulares).

Posteriormente, añadimos capas de complejidad progresiva mediante `@media` a medida que el espacio en pantalla aumenta.

- **Ventajas de rendimiento:** Los teléfonos móviles suelen tener menos potencia de hardware y conexiones a internet más lentas que las computadoras de escritorio. Al escribir los estilos móviles por defecto, evitamos que los celulares tengan que parsear e invalidar reglas de escritorio pesadas de CSS. Además, esto nos obliga a priorizar el contenido y la jerarquía de la información desde el inicio del proyecto.

- **min-width frente a max-width:**

- La estrategia Mobile First se implementa utilizando media queries con min-width (por ejemplo, `@media (min-width: 768px)` significa “a partir de 768px de ancho en adelante”).
- Si usáramos max-width, estaríamos escribiendo estilos para pantallas grandes y luego “rompiéndolos” o sobrescribiéndolos para pantallas chicas, lo que genera código repetitivo, overrides constantes y hojas de estilo infladas difíciles de mantener.

- **grid-template-columns: repeat(2, 1fr):**

- En tabletas, modificamos nuestras grillas a dos columnas iguales.
- **La unidad 1fr (Fractional Unit):** Es una unidad nativa de CSS Grid que representa una fracción del espacio libre disponible dentro del contenedor. El uso de `repeat(2, 1fr)` calcula automáticamente el ancho al 50 % para cada columna restando los espacios definidos por gap, lo cual simplifica los cálculos matemáticos manuales que antes se hacían con `width: calc(50% - gap);`.

- **Propiedad order en Flexbox:**

- En pantallas medianas y grandes, utilizamos la propiedad order para reordenar la disposición visual de los filtros: `.filtros-categorias { order: 1 }, .filtros-buscador { order: 2 }, y .filtros-contador { order: 3 }.`
- **Cómo funciona:** Por defecto, todos los elementos en un contenedor flex tienen un valor de `order: 0` y se posicionan según su orden secuencial en el código HTML. Al darles números de orden progresivos (1, 2, 3), podemos alterar visualmente su posición en pantalla sin necesidad de tocar la estructura del DOM en HTML.

- **[!WARNING]**

**Advertencia de Accesibilidad:** Cambiar demasiado el orden visual con order sin alterar el DOM puede generar confusión en personas ciegas que usan lectores de pantalla (ya que escucharán el contenido en el orden original del HTML) o en usuarios que navegan usando la tecla Tabulador (pues el foco saltará de manera desordenada por la pantalla). Usa order con prudencia y sólo para ajustes menores de diseño.

- **.menu-hamburguesa { display: none }:** En tabletas y computadoras ocultamos el botón hamburguesa móvil (`display: none`), ya que el espacio es suficiente para mostrar la barra de navegación completa horizontal expandida de forma permanente.
- **flex-direction: row en newsletter:** En móvil, el input de correo y el botón se apilan verticalmente (`column`) para que sean cómodos de presionar en celulares; en pantallas más anchas los colocamos de forma horizontal (`row`) para aprovechar el espacio a lo ancho de la pantalla de manera elegante.
- **repeat(4, 1fr) en desktop:** Las tarjetas de “¿Por qué Santa Cruz?” pasan a 4 columnas. El footer también, distribuyendo los bloques de navegación, información y redes sociales de forma armónica a lo largo de toda la pantalla de escritorio.

## Paso 10.2: Modo oscuro automático

```
@media (prefers-color-scheme: dark) {
  :root {
    --color-fondo: #121212;
    --color-superficie: #1e1e1e;
    --color-texto: #e8e6e1;
    --color-texto-secundario: #a8a6a1;
    --color-borde: #333333;
    --color-nieve: #181818;
    --color-cielo: #2c3e50;
    --sombra-sm: 0 2px 4px rgba(0, 0, 0, 0.3);
    --sombra-md: 0 4px 12px rgba(0, 0, 0, 0.4);
    --sombra-lg: 0 12px 30px rgba(0, 0, 0, 0.5);
  }

  img {
    filter: brightness(0.9) contrast(1.1);
  }

  .seccion-titulo { color: var(--color-glaciar); }
  .tarjeta-razon-titulo { color: var(--color-glaciar); }
  .tarjeta-destino-nombre { color: var(--color-glaciar); }
```

```
.testimonio-nombre { color: var(--color-glaciar); }
.contacto-seccion-titulo { color: var(--color-glaciar); }
}
```

### Decisiones de implementación, línea por línea:

- **@media (prefers-color-scheme: dark):** Media query de accesibilidad de CSS que detecta automáticamente las preferencias de tema del sistema operativo o del agente de usuario (si el usuario tiene activado el “Modo Oscuro” en Windows, macOS, Android, iOS o en la configuración de su navegador web). Esto permite una transición nativa y automática sin obligar al usuario a hacer clic en un switch manual.
- **La gran ventaja de las CSS Custom Properties (Variables):**
  - En la maquetación tradicional anterior a CSS3, para dar soporte a modo oscuro había que duplicar casi todas las reglas del sitio agregando clases (por ejemplo: `.tarjeta { background: #fff }` y `.dark-theme .tarjeta { background: #1e1e1e }`).
  - Al utilizar variables CSS declaradas globalmente en `:root`, **únicamente redefinimos los valores de las variables de color** dentro de la media query de modo oscuro. Como todo nuestro archivo CSS base ya consume estas variables (`var(--color-fondo)`, `var(--color-texto)`, etc.), el navegador recalcula e inyecta los nuevos colores de forma instantánea en todos los elementos del DOM. Esto reduce el tamaño de nuestra hoja de estilos y previene errores de “olvidar cambiar el color de un elemento”.
- **filter: brightness(0.9) contrast(1.1) en imágenes:**
  - En entornos oscuros, las imágenes de fondos claros o muy iluminadas (como un glaciar blanco con sol) pueden generar destellos de luz molestos para la vista (“glare”).
  - Para evitar la fatiga visual del usuario, aplicamos una regla general a todos los elementos `img` disminuyendo su brillo al 90 % (`brightness(0.9)`). Para compensar la ligera pérdida de nitidez al oscurecerlas, incrementamos su contraste en un 10 % (`contrast(1.1)`). Esto integra las imágenes de manera suave y armónica con la estética oscura de la interfaz general.
- **Ajustes puntuales de contraste y color:**
  - Títulos de sección como `.seccion-titulo`, `.tarjeta-destino-nombre`, etc., tenían configurado el color azul profundo `--color-lago (#0f2a4a)` en el modo claro por defecto.
  - En un fondo negro o gris muy oscuro, el azul marino profundo tiene bajísimo contraste, violando las pautas de accesibilidad WCAG de legibilidad de texto. Por ello, sobrescribimos selectivamente el color de estos títulos al azul glaciar (`var(--color-glaciar)`, `#2e86ab`), que es considerablemente más brillante y legible sobre fondos oscuros.

## 13. Arquitectura final: decisiones clave

### 13.1 Separación de responsabilidades

| Capa                    | Rol                                                         | Archivo                     |
|-------------------------|-------------------------------------------------------------|-----------------------------|
| <b>Estructura</b>       | Define qué elementos existen y su semántica                 | <code>index.html</code>     |
| <b>Presentación</b>     | Define cómo se ven esos elementos y su disposición espacial | <code>css/styles.css</code> |
| <b>Conducta y datos</b> | Qué hace la página, cómo gestiona la información            | <code>js/main.js</code>     |

### 13.2 Estado + Renderizado: un flujo unidireccional

El filtrado de destinos adopta un flujo **unidireccional**:

El usuario interactúa (clic en filtro o escritura en buscador)

↓

Se modifican las variables de estado (categoriaActiva, busquedaTexto)

↓

`filtrarDestinos()` actualiza la vista (muestra/oculta tarjetas, contador, mensaje)

Ventajas:

- **Predecibilidad:** siempre sabés dónde está la verdad — en las variables `categoriaActiva` y `busquedaTexto`.
- **Depuración fácil:** un `console.log({ categoriaActiva, busquedaTexto })` en cualquier momento te muestra el estado real.
- **Extensibilidad:** si mañana querés agregar un filtro por “dificultad del sendero”, agregás una variable `dificultadActiva` y una línea en `filtrarDestinos()`. Nada más.

### 13.3 Mobile First y Progressive Enhancement

Diseñamos primero para móvil y luego agrandamos. Esto garantiza que:

- El sitio funciona en CUALQUIER dispositivo, desde un celular antiguo hasta un monitor 4K.
- No dependemos de características avanzadas que puedan no estar disponibles.
- La experiencia base es sólida; las mejoras (modo oscuro, animaciones) son “capas” opcionales.

### 13.4 Accesibilidad como parte del diseño

No agregamos ARIA “al final”. Desde la Tarea 1, cada elemento interactivo tiene:

- `aria-label` para elementos sin texto descriptivo.
- `aria-expanded` para el menú hamburguesa.
- `aria-pressed` para botones de filtro.
- `aria-invalid` + `aria-describedby` para validación de formularios.
- `role="alert"` + `aria-live` para feedback dinámico.
- `aria-hidden="true"` para contenido decorativo.

Esto no es “extra”: es parte de la funcionalidad. Un sitio sin accesibilidad no está completo.

### 13.5 Por qué no usamos frameworks

Este proyecto es 100 % vanilla (HTML + CSS + JS) porque:

- **Aprendizaje:** entender cómo funcionan las cosas “de abajo” te hace mejor desarrollador cuando sí usás frameworks.
- **Simplicidad:** no hay build tools, no hay dependencias, no hay `package.json`. Se abre directamente en el navegador.
- **Performance:** sin librerías que descargar, la página carga instantáneamente.

## 14. Checklist de aprendizaje

Al finalizar esta guía, el alumno debería ser capaz de explicar y reproducir:

- ☐ Estructura semántica de un documento HTML5 con `header`, `nav`, `main`, `section`, `article`, `footer`, `address`.
- ☐ Uso de atributos `data-*` para almacenar metadatos en elementos HTML.
- ☐ Maquetación con CSS Grid para grillas de tarjetas (1, 2, 3, 4 columnas según pantalla).
- ☐ Maquetación con CSS Flexbox para distribución en una dimensión (header, formularios).
- ☐ CSS Custom Properties (variables) para mantener una paleta de colores coherente.
- ☐ Patrón Mobile First: estilos base para móvil, media queries `min-width` para agrandar.
- ☐ Modo oscuro automático con `prefers-color-scheme` redefiniendo variables.
- ☐ Delegación de eventos vs listeners individuales: cuándo usar cada uno.
- ☐ Manejo de estado con variables (`categoriaActiva`, `busquedaTexto`) y función de renderizado (`filtrarDestinos`).
- ☐ Normalización de strings para búsqueda case-insensitive y sin acentos.
- ☐ Validación de formularios con expresiones regulares y feedback accesible (`aria-invalid`).
- ☐ Navegación por hash sin recargar (`history.pushState`, `hashchange`, `scrollIntoView`).
- ☐ Menú hamburguesa con transiciones CSS y atributos ARIA dinámicos.
- ☐ Lazy loading de imágenes con `loading="lazy"`.
- ☐ Sticky header y sticky footer con `position: sticky` y `margin-top: auto`.

## Apéndice: estructura de archivos final

`portal-turismo-simple/`

- |— index.html → Estructura y contenido de la landing page
- |— css/
  - |— styles.css → Reglas de presentación, layout responsive, modo oscuro
- |— js/
  - |— main.js → Lógica: menú, filtrado, validación, navegación
- |— img/
  - |— hero-santa-cruz.jpg
  - |— glaciario-perito-moreno.jpg
  - |— monte-fitz-roy.jpg
  - |— cueva-de-las-manos.jpg
  - |— bosque-petrificado.jpg
  - |— lago-argentino.jpg
  - |— puerto-deseado.jpg
  - |— rio-pinturas.jpg
  - |— estancias-patagonicas.jpg

**Consejo para practicar.** Replicá el proyecto desde cero con esta guía cerrada, y abrila solo cuando te bloquee. La repetición activa —“hacer sin mirar”— es la que consolida el aprendizaje. Y un ejercicio extra muy recomendable: una vez que tu versión funcione, intentá explicarle en voz alta a un compañero **por qué filtrarDestinos es una sola función que lee del estado**. Si podés explicarlo, lo entendiste.